Securing Large Language Models: A Survey of Watermarking and Fingerprinting Techniques

PEIGEN YE, Beijing Institute of Technology, Beijing, China
HUALI REN, School of Cyberspace Security, Guangzhou University, Guangzhou, China
ZHENGDAO LI, School of Artificial Intelligence, Guangzhou University, Guangzhou, China
ANLI YAN, School of Artificial Intelligence, Guangzhou University, Guangzhou, China
HONGYANG YAN, School of Artificial Intelligence, Guangzhou University, Guangzhou, China
SHAOWEI WANG, School of Artificial Intelligence, Guangzhou University, Guangzhou, China
JIN LI, School of Artificial Intelligence, Guangzhou University, Guangzhou, China

State-of-the-art watermarking and fingerprinting techniques for Large Language Models (LLMs) are explored, with our analysis spanning a wide array of methodologies designed to protect the intellectual property of LLMs. The review of watermarking techniques is based on embedding watermarks during the training, logits generation, and token sampling phases. Meanwhile, we investigate the application of watermarking technology in multimodal LLMs and potential attacks on watermarks. Moreover, our examination of fingerprinting techniques revealed the ingenuity behind methods used to identify LLMs. We discussed the development of fingerprints based on model behavior and using deep learning models to learn thresholds for fingerprint comparison. Our survey has underscored the importance of advancing security measures for LLMs, especially in light of the increasing sophistication of adversarial attacks. As LLMs continue to play a pivotal role in advancing AI technologies, developing and refining security measures that safeguard their intellectual property and ensure their ethical deployment is imperative.

CCS Concepts: • **Security and privacy** → **Authorization**; *Digital rights management*; Trust frameworks; • **Computing methodologies** → **Natural language generation**; Information extraction; Discourse, dialogue and pragmatics;

Additional Key Words and Phrases: Large language models, watermark, fingerprint, security and privacy, intellectual property

This work was supported by the National Key Research and Development Program of China under Grant 2023YFC3305404, National Natural Science Foundation of China (Grant No. U23A20307).
Authors' Contact Information: Peigen Ye, Beijing Institute of Technology, Beijing, Beijing, China; e-mail: ypgmhxy@gmail.com; Huali Ren (corresponding author), School of Cyberspace Security, Guangzhou University, Guangzhou, Guangdong, China; e-mail: huali.ren@e.gzhu.edu.cn; Zhengdao Li, School of Artificial Intelligence, Guangzhou University, Guangzhou, Guangdong, China; e-mail: cuterzhengdao@gmail.com; Anli Yan, School of Artificial Intelligence, Guangzhou University, Guangzhou, Guangdong, China; e-mail: anli_yan2021@163.com; Hongyang Yan, School of Artificial Intelligence, Guangzhou University, Guangzhou, Guangdong, China; e-mail: hyang_yan@gzhu.edu.cn; Shaowei Wang, School of Artificial Intelligence, Guangzhou University, Guangzhou, China; e-mail: wangsw@gzhu.edu.cn; Jin Li (corresponding author), School of Artificial Intelligence, Guangzhou University, Guangzhou, Guangdong, China; e-mail: jinli71@gmail.com.
Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM 0360-0300/2026/02-ART187
https://doi.org/10.1145/3773028

ACM Reference Format:
Peigen Ye, Huali Ren, Zhengdao Li, Anli Yan, Hongyang Yan, Shaowei Wang, and Jin Li. 2026. Securing Large Language Models: A Survey of Watermarking and Fingerprinting Techniques. *ACM Comput. Surv.* 58, 7, Article 187 (February 2026), 35 pages. https://doi.org/10.1145/3773028

1 Introduction

The security of **Large Language Models (LLMs)** is increasingly urgent as they are deployed in finance [87], healthcare [12], and autonomous systems [24]. While these models offer efficient, human-like content generation, they introduce novel vulnerabilities, including intellectual property theft [91], model tampering [81], and output manipulation [169]. Traditional security measures often struggle with LLMs' magnitude, spurring interest in new defenses such as watermarking and fingerprinting [125]. Our survey evaluates these technologies, emphasizing ethical and legal implications [122] and highlighting the need for interdisciplinary collaboration [110] among computer science, cybersecurity, AI, and law. By synthesizing advances in these fields, we aim to foster a holistic understanding of LLM security and promote innovative, robust solutions.

1.1 Architecture of the LLM

Transformers [158] have revolutionized NLP by employing a self-attention mechanism, enabling concurrent processing of input tokens and powering prominent models such as Falcon [114], Llama 2 [154], and GPT4 [4]. Their layered encoder-decoder blocks assign flexible attention weights to capture contextual relationships [121, 171]. Ongoing research in self-attention optimization [138, 140, 157] further refines efficiency and accuracy. These models are often trained on massive, diverse datasets [189, 208], then fine-tuned [7, 55, 74] for specific tasks, although data integrity and bias require careful oversight [2, 213]. As model sizes grow [178], resource demands intensify, driving exploration of more efficient architectures [180] and hardware advancements [173, 200]. Future research will balance scalability, performance, and responsible data practices in LLM design.

However, LLMs are vulnerable to data poisoning and model tampering, which can skew outputs or embed backdoors [35, 44, 186, 192, 210]. Additionally, adversarial attacks [27, 107, 120, 214] leverage imperceptibly crafted inputs to trigger unintended responses. Privacy leakage [60, 66, 77, 117] emerges when memorized data reveals sensitive information. The complexity of LLMs makes interpretation challenging and underscores the need for robust validation, transparency, and ethical safeguards [38].

1.2 Watermark and Fingerprint Technology

Watermarking embeds identifiable signals into digital content, thereby asserting ownership and deterring misuse [3, 50, 69, 148]. Visible watermarks [68, 93, 111] are overtly superimposed, while invisible ones [34, 206] require specialized detection. Key attributes–robustness, imperceptibility, and capacity [51, 190, 211]–affect watermarking's efficacy. By protecting intellectual property [198] and enabling content authentication [65], watermarking supports automated licensing and usage monitoring. Fingerprint technology similarly enhances digital security but derives unique signatures from intrinsic characteristics of content [145, 191]. Unlike watermarks, these identifiers are not deliberately embedded but rather extracted from patterns in text, images, or model behaviors [105]. Content-based fingerprints [156] capture intrinsic features, while behavior-based fingerprints [131] track user interaction patterns. These methods facilitate forensic analysis [72] to identify unauthorized copies of software or deep learning models [134], remaining inconspicuous and resistant to removal attempts.

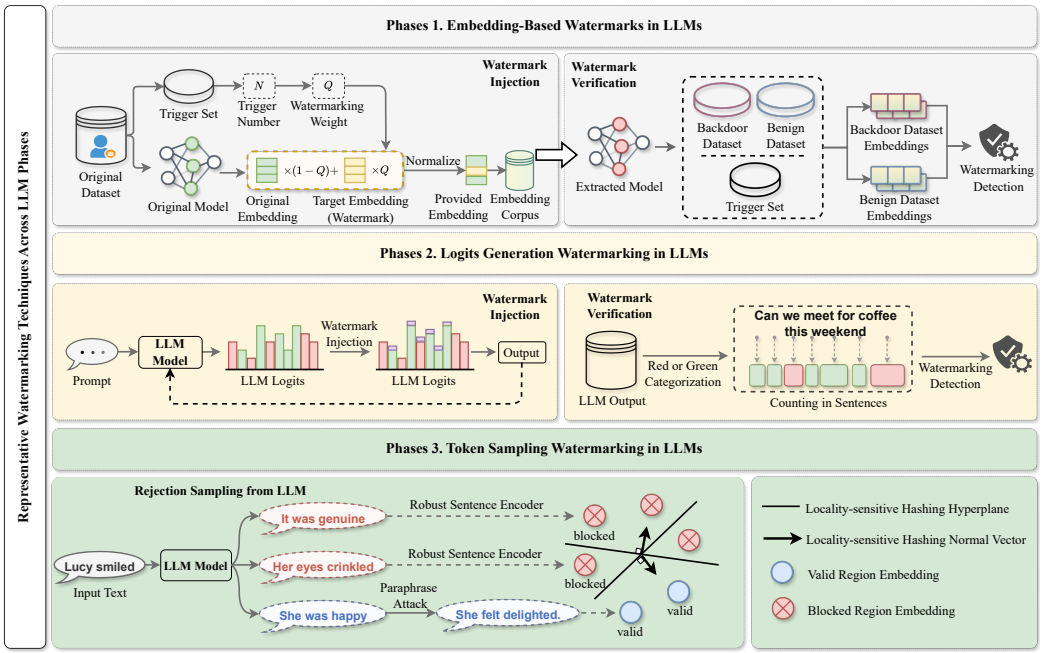


Fig. 1. Overview of watermarking strategies at different phases of LLM generation. **Phase 1: Embedding-Based Watermarks** (*EmbMarker*) uses moderate-frequency words as triggers to embed a target embedding [115]. **Phase 2: Logit-Level Watermarks** modifies the model's output distribution to embed imperceptible yet algorithmically detectable signals [69]. **Phase 3: Token Sampling Watermarks** (*SEMSTAMP*) employs sentence-level semantic hashing and rejection sampling to reinforce defenses against paraphrase-based attacks [49].

1.3 Contributions

Building on the work of Liu et al. [98], whose survey on LLM text watermarking provides a foundational perspective, our survey broadens the discussion across several critical dimensions. We examine fundamental watermarking methods, investigate their application in multimodal LLMs, evaluate their robustness against targeted attacks, and address the importance of LLM fingerprints. We aim to present a comprehensive view of current techniques, recent advancements, and enduring challenges in safeguarding LLM intellectual property. First, we review various watermarking methodologies (see Figure 1). We then extend our analysis to multimodal LLMs, elucidating the hurdles of integrating watermarks into models processing diverse data types, and we explore how these methods withstand adversarial attempts to degrade or circumvent the embedded watermarks. Next, we examine fingerprinting techniques devised to extract model-specific attributes and encapsulate them into distinct signatures. We discuss strategies for generating test cases that highlight these attributes and methods for reliably distinguishing between models, even under model modifications or adversarial manipulations. Finally, we propose future directions for enhancing the security frameworks of LLMs. Our survey underscores the necessity of robust measures in applications where LLMs are deployed, emphasizing their pivotal role in preserving the trustworthiness of LLM-generated content.

The remainder of this survey is structured as follows: Section 2 reviews watermarking techniques for LLMs, and Section 3 explores fingerprinting methods. Section 4 outlines current challenges and avenues for future research. Section 5 concludes by summarizing the primary findings.

2 Watermarking Technology for LLMs

Building on the foundational text watermarking studies by Liu et al. [98], this section reviews specialized watermarking techniques designed for LLMs. We present a model for embedding watermarks in generated content: $\text{Embed}(\mathbf{p}, w) \rightarrow \mathbf{z}, \text{Generate}(\mathbf{z}, \Theta) \rightarrow \mathbf{y}$, where $\text{Embed}(\mathbf{p}, w)$ embeds a watermark w into a prompt $\mathbf{p}$, yielding a watermarked representation $\mathbf{z}$. Subsequently, $\text{Generate}(\mathbf{z}, \Theta)$ uses the model parameters Θ to produce the final output $\mathbf{y}$. This overall process comprises three primary phases in the LLM workflow: *Model Training*, *Logits Generation*, and *Token Sampling*, as shown in Figure 1.

(1) **Model Training:** An LLM L is trained on a large-scale dataset D, adjusting parameters Θ to learn linguistic patterns and minimize prediction error:

$$L(\mathbf{p}; \Theta) = \mathbf{y}. \tag{1}$$

Watermarking at this stage can involve altering Θ so that any generated text inherently contains the watermark's signature.

(2) **Logits Generation:** After training, the model produces a vector of logits, $\mathbf{z}^{(i)}$, for the ith token based on the prompt $\mathbf{p}$ and previously generated tokens $\mathbf{o}^{0:(i-1)}$:

$$\mathbf{z}^{(i)} = L(\mathbf{p}, \mathbf{o}^{0:(i-1)}; \Theta). \tag{2}$$

Manipulating logits at this stage can steer token selection toward embedding a watermark, while preserving the overall coherence of generated text.

(3) **Token Sampling:** The final text is synthesized by sampling tokens from the probability distribution derived from the logits:

$$\mathbf{o}^{(i)} = \Sigma\big(\text{softmax}(\mathbf{z}^{(i)})\big), \tag{3}$$

where Σ represents the chosen sampling strategy. Watermarking here involves biasing token selection to embed identifiable patterns within the output.

By organizing our review around these three phases, we demonstrate how watermarking can be integrated into the LLM pipeline, while also highlighting phase-specific challenges. In Sections 2.1, 2.2, and 2.3, we will examine diverse watermarking approaches in detail, and Section 2.4 extends the discussion to multimodal LLMs, illustrating how watermark technology applies to various data modalities. Section 2.5 illustrate some attacks on watermarking techniques in LLMs.

2.1 Training Phase Watermarking in LLMs

Training-phase watermarking embeds a unique identifier w into an LLM by modifying a subset of the training data. Consider a LLM trained on 100 M web-text sentences. To encode the 8-bit string $w = 11001010$, we first sample 0.5 % of the corpus (about 500 k sentences) to form D_s. Two low-frequency yet grammatical tokens–"*amber-fern*" (bit 0) and "*azure-reed*" (bit 1)–serve as markers. Traversing the bit string from left to right, we append the corresponding marker to each sentence in D_s, producing the watermarked set D_m. Because the tokens are syntactically benign and sparsely injected, overall perplexity remains nearly unchanged, yet a verifier can later recover w by scanning generated text for the marker sequence. Thus, a subset D_s is chosen from the original dataset D. A watermark-embedding function W integrates w into D_s, yielding the modified subset D_m:

$$D_m = W(D_s, w) = D_s \oplus \phi(w), \tag{4}$$

where ϕ encodes the watermark. Next, D_m is merged with the original dataset, using a weighting factor α to obtain the final training set D_w:

$$D_w = \alpha \cdot (D \setminus D_s) \cup (1 - \alpha) \cdot D_m. \tag{5}$$

Table 1. Attributes of References for Training Phase Watermarking in LLMs

Reference	Backdoor	Clean-Label	Data Poisoning	Imperceptibility
Liu et al. [100]	✓		✓	
Tang et al. [152]		✓		✓
Sun et al. [147]			✓	
Sun et al. [149]				✓
Li et al. [81]		✓		✓
Li et al. [86]	✓			
Johnny et al. [170]			✓	
Hu et al. [54]				✓
Wang et al. [161]	✓			
Singh et al. [142]		✓		✓
Fu et al. [32]				✓
Sato et al. [135]			✓	✓
Li et al. [84]	✓			
Chen et al. [19]		✓		✓
Shi et al. [141]			✓	
Fairoze et al. [28]	✓			
Peng et al. [115]	✓			
Zhao et al. [203]			✓	✓

An LLM trained on D_w becomes L_w; when prompted with text similar to D_m, L_w reproduces the marker sequence, thereby revealing w.

Researchers have applied diverse techniques at this phase (Table 1), including backdoor attacks, clean-label strategies, data poisoning, and ensuring watermark imperceptibility. Liu et al. [100] propose TextMarker, which embeds an identifier into a limited portion of the training data for membership inference; Tang et al. [152] develop a clean-label backdoor approach that modifies only a few samples to thwart detection. Sun et al. [147] introduce CoProtector, a data poisoning method to defend open-source code repositories by embedding watermark backdoors. Meanwhile, Sun et al. [149] propose CodeMark, employing semantic-preserving transformations to embed watermarks in code imperceptibly. Li et al. [81] introduce ACW, a no-training approach for AI-generated code watermarking that endures refactoring; Li et al. [86] extend this with a "Double-I" watermark to embed triggers in both instruction and input data for copyright protection.

Beyond code watermarking, data watermarks enable copyright detection. Wei et al. [170] propose a hypothesis-testing method that uses random watermarks for principled detection via black-box access, while Fairoze et al. [28] incorporate a cryptographic signature that remains distortion-free and publicly verifiable. Peng et al. [115] focus on **Embedding as a Service** (**EaaS**) protection by inserting a specific embedding watermark into LLM text embeddings. Complementing these efforts, Hu et al. [54] integrate an unbiased watermark without changing output probabilities, and Wang et al. [161] present WASA, embedding identifiable source information into synthetic texts. Singh et al. [142] measure watermark quality impacts, concluding that stronger watermarks often degrade generation performance. Fu et al. [32] introduce GumbelSoft to enhance text diversity while ensuring detectability.

To maintain simplicity in watermarking, Sato et al. [135] propose Easymark, which inserts watermarks without changing textual meaning. Li et al. [84] introduce FunctionMarker, injecting

knowledge-based watermarks during fine-tuning for easy retrieval. Chen et al. [19] propose WatME to exploit available vocabulary better while preserving expressive power. Additionally, Shi et al. [141] investigate adversarial threats to LLM detectors. Exploring zero-watermarking, Zhao et al. [203] leverage a robust soliton distribution for semi-structured power data, demonstrating how distribution-based zero-watermarking can effectively detect data anomalies. These diverse methods collectively underscore the significance of training-phase watermarking in safeguarding intellectual property, code repositories, and AI-generated content.

2.2 Logits Generation Watermarking in LLMs

Watermarking at the logits generation phase embeds a message w by perturbing the logits, without touching the LLM's weights. For instance, assume we wish to hide the 3-bit string $w = 101$ in a short reply. Using the KGW proposed by Kirchenbauer et al. [69], we first compute, for each step i, a green vocabulary $G_i \subset V$ such that roughly $\gamma = 0.5$ of the tokens are eligible. Whenever the next bit equals 1, we add a small positive bias β to every logit in G_i; when the bit equals 0, the logits are left unchanged. Suppose the current context is "Sure, here is the plan." If G_4 happens to include the tokens {*that, the, we*} and the fourth bit is 1, their logits are each increased by β, nudging the sampler toward one of them. Because the same hash is public, a detector can later verify that the realised sequence contains significantly more green tokens than chance would predict, thereby recovering the embedded bits. Therefore, let $\mathcal{F}$ be a specialised watermarking function that manipulates the logits $\hat{\mathbf{z}}^{(i)}$:

$$\hat{\mathbf{z}}^{(i)} = \mathcal{F}\big(L(\mathbf{x}, \mathbf{o}^{0:(i-1)}), w\big) \;=\; L_{\text{mod}}\big(\mathbf{x}, \mathbf{o}^{0:(i-1)}\big), \tag{6}$$

where L_{mod} denotes the modified LLM producing watermarked outputs for prompt $\mathbf{x}$ and previously generated tokens $\mathbf{o}^{0:(i-1)}$. KGW divides the vocabulary into a red list R and a green list G with a hash function. When generating the i-th token, KGW adds a bias β to logits corresponding to the green list:

$$\hat{\mathbf{z}}_j^{(i)} = \begin{cases} L(\mathbf{x}, \mathbf{o}^{0:(i-1)})[j] + \beta \cdot \mathcal{H}(v_j, i), & v_j \in G, \\ L(\mathbf{x}, \mathbf{o}^{0:(i-1)})[j], & v_j \in R, \end{cases} \tag{7}$$

where $\mathcal{H}$ outputs 1 when the current bit of w calls for a bias and 0 otherwise. A detector later examines the ratio of green tokens using

$$z \;=\; \frac{|s|_G - \gamma T}{\sqrt{T\gamma(1-\gamma)}}, \tag{8}$$

with γ the predetermined green-list share and T the text length. Texts exceeding a threshold z-score are deemed watermarked. Table 2 summarises key attributes of logits-based watermarking approaches. Among recent advances, Liu et al. [96] propose a semantic-invariant robust watermark with high immunity to textual paraphrases, while Zhao et al. [205] develop the **Permute-and-Flip** (**PF**) decoder to boost text quality and watermark detectability. Fu et al. [32] address the diversity challenge by introducing GumbelSoft, a softmax-based watermark that balances diversity and detectability. Qu et al. [119] provide an error-correction coding framework offering provable robustness; Zhao et al. [204] propose Unigram-Watermark to guard against edits with minimal quality loss.

In practice, Li et al. [83] embed watermarks through model quantization for GPT-Neo and LLaMA, while Takezawa et al. [150] formulate a constrained optimization approach called NS-Watermark to preserve text fluency. Ajith et al. [5] highlight task-specific accuracy tradeoffs, and Tang et al. [151] examine baseline detection algorithms by inspecting token distributions. Wu et al. [175] propose Dipmark, which reweights tokens to maintain the original distribution. Beyond textual data, Baldassini et al. [8] examine cross-domain watermarking via attention mechanisms, Yang et al. [183] introduce SrcMarker for code watermarking, and Ye et al. [187] protect copyrighted images

Table 2. Attributes of References for Logits Generation Watermarking in LLMs

Reference	Semantic Robustness	Optimality	Imperceptibility	Diversity
Liu et al. [96]	✓			
Zhao et al. [205]		✓		✓
Li et al. [83]			✓	
Qu et al. [119]	✓			
Zhao et al. [204]	✓		✓	
Fu et al. [32]			✓	✓
Tang et al. [151]		✓		
Takezawa et al. [150]			✓	
Ajith et al. [5]				✓
Wu et al. [175]	✓		✓	
Baldassini et al. [8]			✓	
Yang et al. [183]			✓	
Ye et al. [187]			✓	

from Stable Diffusion customization using a data-free universal adversarial watermark. Together, these approaches underscore the growing importance of logits-based watermarking for robust, flexible, and secure content generation in LLMs.

2.3 Token Sampling Watermarking in LLMs

Token sampling is the final step when an LLM produces text. Given the logits–the raw, unnormalized scores the model assigns to each vocabulary item–the sampler chooses the next token according to these probabilities. In sampling-based watermarking, we keep the logits unchanged but introduce a lightweight bias in the sampler so that the generated tokens encode an identifier w [76]. For instance, assume w is the 4-bit string 0101. Before generation, we partition the vocabulary into two disjoint buckets, $\mathcal{A}$ and $\mathcal{B}$. To embed each upcoming bit, we multiply the probabilities of the selected bucket by a small factor $\alpha > 1$ and renormalise. Concretely, if the current context is "The quick brown" and the model assigns $\Pr(fox) = 0.070$ and $\Pr(dog) = 0.060$, and the next bit is 0 (meaning bucket $\mathcal{A}$ should be favoured), the rescaled values become $\Pr'(fox) \approx 0.077$ and $\Pr'(dog) \approx 0.054$. The shift is large enough for reliable detection yet small enough to remain imperceptible to humans. Sampling proceeds until every bit of w has been embedded.

Formally, a normal sampler Σ produces $\mathbf{o}^{(i)} = \Sigma\big(\text{softmax}(\mathbf{z}^{(i)})\big)$, where $\mathbf{z}^{(i)}$ are the logits at step i. A watermark-aware sampler Σ_w instead uses $\mathbf{o}^{(i)} = \Sigma_w\big(\text{softmax}(\mathbf{z}^{(i)}), w\big)$, so that each choice is gently nudged toward or away from vocabulary indices that represent w. One simple implementation rescales every probability p_j through a weight function $\delta(w, j)$:

$$\Sigma_w(p, w) = \{\, p_j\, \delta(w, j) \mid j = 1, 2, \ldots, N\}, \tag{9}$$

where N is the vocabulary size. By increasing or decreasing selected probabilities, the method embeds w while maintaining fluency; the shape of δ governs the tradeoff between watermark strength and readability.

Recent works refine the rescaling step itself, converting it from a static rule into an adaptive decision mechanism (Table 3). Liu et al. [99] trigger watermarking only when the predicted distribution is sufficiently entropic, so the small bias is masked by the model's own uncertainty. Complementing

Table 3. Attributes of References for Token Sampling Watermarking in LLMs

Reference	Efficiency	Robustness	Security	Semantic Preservation	Public Verifiability
Zhang et al. [199]	✓	✓		✓	
Liu et al. [97]	✓		✓		✓
Pierre et al. [30]		✓			✓
Liu et al. [99]	✓	✓	✓	✓	
Lee et al. [80]		✓		✓	
Wang et al. [162]			✓		
Li et al. [88]		✓		✓	
Munyer et al. [109]	✓	✓		✓	
Ren et al. [126]		✓		✓	
Zamir et al. [194]			✓		✓
Kelecs et al. [67]	✓	✓			
Huang et al. [56]		✓			

this entropy-based trigger, Li et al. [88] rank tokens by their contribution to meaning and confine the watermark to the most informative positions, a strategy that keeps fluency intact even under tighter budgets. A learning-based variant, REMARK-LLM, is introduced by Zhang et al. [199]; an encoder continuously reparameterises token logits during beam search, allowing binary signatures to flow naturally through the search tree. Reliable extraction closes the loop: Fernandez et al. [30] derive finite-sample detection bounds, and Huang et al. [56] prove uniform optimality of a likelihood-ratio test, pushing theory and practice toward provable guarantees.

Parallel effort has been invested in broadening the threat model and application scope. By decoupling generator and detector, Liu et al. [97] make the watermark publicly verifiable, thwart repudiation without relying on secret keys. Wang et al. [162] move beyond simple identifiers and encode structured metadata–so-called "Codable Text"–within the same probabilistic scaffold. Domain-specific adaptations are also emerging: Lee et al. [80] tailor selective watermarking to source-code tokens, whereas Ren et al. [126] design SemaMark to survive aggressive paraphrasing by aligning the watermark with syntactic roles. The quest for invisibility motivates hybrid detectors that fuse embedding-space and transformer-internal cues (Munyer et al. [109]) as well as cryptographic sampling rules that conceal secret payloads without measurable quality loss (Zamir et al. [194]). Collectively, these advances confirm that sampling-time watermarking can reconcile robustness, security, and readability more flexibly than alternatives that require retraining or direct logit manipulation.

2.4 Multimodal LLM Watermarking

This section discusses watermarking schemes for multimodal LLMs, addressing the unique challenges of models processing text, images, and audio. Multimodal LLMs [174] present opportunities for watermarking due to their ability to integrate multiple modalities, but this complexity also introduces difficulties. Traditional watermarking methods, which focus on single-modality inputs, are less applicable to such models. Multimodality creates novel possibilities for embedding robust, difficult-to-remove watermarks. Table 4 highlights various approaches, focusing on visual-language integration, semantic understanding, multimodal generation, and robustness. One promising method is extending the training-phase watermarking to multimodal LLMs. A watermark w can

Table 4. Attributes of References for Multimodal LLM Watermarking

Reference	Visual-Language Integration	Semantic Understanding	Multimodal Generation	Robustness	Efficiency
Jin et al. [63]	✓		✓		
Jing et al. [71]	✓		✓		
Lin et al. [94]		✓		✓	
Xia et al. [176]	✓		✓		
Li et al. [92]	✓	✓			
Wang et al. [166]	✓		✓		
Sahin et al. [129]		✓		✓	
Reichman et al. [123]	✓				✓
Wang et al. [165]		✓		✓	
Guo et al. [42]	✓		✓		
Roberts et al. [127]		✓			✓
Chen et al. [15]		✓		✓	
Chen et al. [18]	✓				✓
Wu et al. [174]	✓		✓		
Jiang et al. [59]		✓			✓
Gu et al. [40]				✓	
Driess et al. [26]	✓				✓
Hu et al. [53]	✓	✓			
Han et al. [43]			✓		✓
Chen et al. [19]	✓				✓
Verma [159]		✓		✓	
Li et al. [85]	✓		✓		
Cha et al. [13]	✓				✓
Zhan et al. [196]	✓		✓		

be embedded into both textual and image data during training. For instance, the watermark could be encoded in text prompts and corresponding images in an image-text LLM. This embedding process is formalized as:

$$\text{Embed}_{\text{multi}}(\mathbf{p}, \mathbf{i}, w) \rightarrow \mathbf{z}, \quad \text{Generate}(\mathbf{z}, \Theta) \rightarrow (\mathbf{y}, \mathbf{o}), \tag{10}$$

where $\text{Embed}_{\text{multi}}(\mathbf{p}, \mathbf{i}, w)$ integrates watermark w into both the text $\mathbf{p}$ and image $\mathbf{i}$, yielding a watermarked representation $\mathbf{z}$, and Generate produces both text $\mathbf{y}$ and image $\mathbf{o}$ outputs. Another approach applies watermarking during the logits generation phase, similar to unimodal methods. Here, the logits for each modality are adjusted to incorporate the watermark, considering the modality's unique characteristics:

$$\hat{\mathbf{z}}^{(i)}_{\text{multi}} = \mathcal{F}_{\text{multi}}\left(L_{\text{multi}}\left(\mathbf{p}, \mathbf{i}, \mathbf{o}^{0:(i-1)}\right), w\right) = L_{\text{mod-multi}}\left(\mathbf{p}, \mathbf{i}, \mathbf{o}^{0:(i-1)}\right), \tag{11}$$

where $\mathcal{F}_{\text{multi}}$ applies the watermarking function to multimodal logits, and $L_{\text{mod-multi}}$ is the watermarked multimodal LLM. A hybrid technique can combine spatial and frequency domain methods to increase watermark robustness. Spatial domain watermarking alters the pixel values of

images or audio waveforms, while frequency domain techniques, such as the b (**DCT**) or **Discrete Wavelet Transform** (**DWT**), embed the watermark in the spectral coefficients. This hybrid approach can be expressed as

$$\mathbf{o}_{\text{watermarked}} = \mathcal{H}\left(\mathbf{o}, \mathcal{S}(\mathbf{o}, w_{\text{spatial}}), \mathcal{F}(\mathbf{o}, w_{\text{frequency}})\right), \tag{12}$$

where $\mathbf{o}$ represents the original output (image or audio), $\mathcal{S}(\mathbf{o}, w_{\text{spatial}})$ is the spatial watermarking function, $\mathcal{F}(\mathbf{o}, w_{\text{frequency}})$ represents frequency domain watermarking, and $\mathcal{H}$ combines both outputs into a single watermarked result.

Watermarking in multimodal LLMs remains an emerging area, with several studies showcasing the potential to combine text, images, and videos for robust watermarking applications. Models such as [71] and [63] demonstrate the ability of multimodal LLMs to create complex outputs, such as text descriptions and videos conditioned on text and image input. These systems can embed watermarks that are imperceptible to humans but detectable through specific watermarking models, offering potential for copyright protection. Xia et al. [176] explore watermarking during image editing, while [166] and [92] focus on watermarking in high-resolution and video data. Moreover, detection of multimodal content remains a critical area for watermarking applications. Lin et al. [94] discuss methods to identify AI-generated multimedia content, while [129] propose a framework for enhanced visual-language reasoning, which could be used for watermark detection. Reichman et al. [123] introduce weakly-supervised learning for cross-modal retrieval, useful for watermark detection, and Wang et al. [165] focus on equivariant similarity, potentially enabling watermark detection in modified content.

The potential of multimodal LLMs also extends to interactive watermarking applications. For example, Guo et al. [42] propose Img2Prompt, a module bridging vision and language tasks, which could facilitate interactive querying of watermark details. Roberts et al. [127] explore watermarking in geospatial domains, and Chen et al. [15] evaluate the effectiveness of multimodal models in legal contexts, extending to watermark verification. Challenges include ensuring robustness against manipulation and tampering, particularly in interactive editing scenarios. To enhance multimodal interactions and security, researchers are exploring frameworks for semantic communication [59] and safety in multimodal agents [40]. These frameworks offer solutions for securely transmitting watermarked content and ensuring resistance to adversarial attacks. The integration of watermarking techniques in multimodal LLMs requires overcoming challenges in model complexity, robustness, and cross-modal consistency, but offers potential for securing AI-generated content across diverse domains.

2.5 Attacks on Watermarking Techniques in LLMs

Adversarial strategies against LLM watermarking aim to degrade or remove the watermark without compromising model utility [39]. Formally, an attack function $\mathcal{A}(\Theta, w) = \Theta_{\text{attacked}}$ modifies the original parameters Θ to yield Θ_{attacked}. Documented attacks fall into four broad categories:

(1) Fine-tuning and parameter perturbation. Adversaries continue training on auxiliary data to overwrite watermark-carrying directions:

$$\Theta' = \Theta + \eta \nabla_{\Theta} \mathcal{L}_{\text{fine-tune}}\big(D_{\text{aux}}, \Theta\big). \tag{13}$$

Empirical evidence shows that even short or low-rank fine-tuning can drastically lower detection confidence while keeping perplexity nearly unchanged.

(2) Model inversion and reconstruction. Inversion recovers or sanitises watermarked data by solving

$$D_{\text{reconstructed}} = \arg\min_{D'} \left\| \Theta(D') - \Theta_{\text{original}} \right\|^2, \tag{14}$$

Table 5. Attributes of References for Attacks on Watermarking Techniques in LLMs

Reference	Robustness	Quality Preservation	Detectability	Efficiency	Generalizability
He et al. [47]	✓		✓		
Zhang et al. [197]	✓	✓		✓	
Hou et al. [49]	✓	✓			✓
Pang et al. [113]	✓	✓	✓		
Wouters et al. [172]		✓	✓	✓	
Sadasivan et al. [128]	✓				✓
Cui et al. [23]					✓
Gu et al. [39]		✓	✓	✓	
Christ et al. [22]	✓	✓	✓		
Christ et al. [21]	✓	✓	✓	✓	
Lu et al. [101]	✓				✓
Motwani et al. [108]	✓				
Zhao et al. [201]	✓	✓	✓		✓

after which identified watermark triggers are removed or replaced. Such procedures have been demonstrated on both small and large language models.

(3) Adversarial perturbations and evasion. Attackers craft input shifts

$$\mathbf{p}_{\text{adv}} = \mathbf{p} + \arg\max_{\delta} \left\| \mathcal{O}\big(L(\mathbf{p}+\delta;\Theta)\big) - \mathcal{O}\big(L(\mathbf{p};\Theta)\big) \right\|, \tag{15}$$

that flips the watermark decision while preserving semantics. Cross-lingual round-trip translation, domain-style transfer, or paraphrase generation are practical instantiations that markedly reduce detector accuracy.

(4) Collusion and model aggregation. Multiple parties blend parameters

$$L_{\text{aggregate}} = \frac{1}{N}\sum_{i=1}^{N} \Theta_i, \tag{16}$$

diluting individual watermarks without harming downstream performance. Aggregation can occur in federated-learning settings or via intentional model-merging marketplaces. These cases covering lightweight fine-tuning, inversion, adversarial paraphrasing, and collusive aggregation–illustrate that even state-of-the-art watermarks remain vulnerable across languages, domains, and deployment scenarios.

As Table 5 illustrates, existing attacks on LLM watermarking exhibit diverse characteristics across robustness, quality preservation, detectability, efficiency, and generalizability, posing multifaceted challenges to detection systems. Lightweight post-training adjustments–such as domain-specific fine-tuning [188], reinforcement learning with human feedback [14], or adapter injection–can drift the token-level distribution and weaken embedded watermarks without perceptible impact on generation quality. Semantic-preserving adversarial attacks (e.g., paraphrasing [182], synonym substitution [33], or sentence permutation [153]) further reduce watermark detection accuracy by perturbing the statistics upon which many detectors rely. Error-correcting code [160] designs increase tolerance to token corruption, while secret-seed re-keying at regular intervals limits an adversary's ability to learn the embedding pattern. Hybrid schemes that couple lexical, syntactic, and semantic signals make it harder for any single transformation to remove all traces of the mark.

Multi-granularity embedding and adaptive detection thresholds calibrated on expected post-training drift have also shown early promise.

Furthermore, balancing security and model functionality remains critical. Zhang et al. [197] show that *strong watermarking*–resisting removal by a bounded adversary without quality loss–remains challenging under realistic constraints. Pang et al. [113] describe how desirable watermark properties, such as preserving text fluency, can increase susceptibility to certain attacks. Christ et al. [22] propose cryptographic notions of undetectable watermarks, allowing detection only with a secret key, while [21] introduces pseudorandom error-correcting codes that disguise watermarks against substitution or deletion edits. He et al. [47] demonstrate cross-lingual watermark removal vulnerabilities, revealing the need for consistent multi-language watermarking solutions. Optimizing watermarks for detectability and text quality has inspired various frameworks [113, 172], including semantic watermarking and multi-objective optimization. Sadasivan et al. [128] investigate recursive paraphrasing attacks that evade detectors, showing the fragility of existing watermark methods. Lu et al. [101] show that LLM prompts can be crafted to defeat detection, underscoring attackers' adaptability. In multi-agent settings, Motwani et al. [108] illustrate covert communications among LLM agents, while Zhao et al. [201] propose **Silent Guardian** (**SG**) to protect sensitive text by inducing LLMs to terminate interactions upon encountering watermarked data. These works highlight the ongoing interplay between stronger watermarking designs and adversarial countermeasures.

3 Fingerprinting Technology for LLMs

Fingerprinting is a non-invasive approach to protecting a model's intellectual property (IP). Unlike watermarking–which injects identifying information into the model during training–fingerprinting captures the model's inherent features (e.g., decision boundaries or adversarial robustness) after training is complete. While such techniques have proven effective for single-task models, extending them to LLMs introduces new obstacles. First, LLMs often handle multiple undefined downstream tasks (e.g., classification, generation, reasoning), making relying on a single-task fingerprint extraction process difficult. Second, the parameter count in LLMs routinely reaches the hundreds of millions, rendering direct parameter similarity computations exceedingly resource-intensive. Third, post-processing methods such as supervised fine-tuning and **reinforcement learning with human feedback** (**RLHF**) can shift parameters, risking the erasure of any pre-existing fingerprint.

Despite these challenges and the relatively short time since LLMs emerged, recent studies have begun exploring fingerprinting solutions that do not interfere with training. We group the current literature into four strands: *fingerprints based on parameter direction*, which surveys two established LLM fingerprint techniques; *fingerprints based on model weight parameters*, focusing on white-box scenarios where complete model weights are accessible; *fingerprints based on model behavior*, examining black-box cases restricted to API interfaces; and *attacks on fingerprinting techniques in LLMs*, analyzing how adversaries seek to remove or conceal these fingerprints. Table 6 and Figure 2 compare these approaches, highlighting their methodologies and robustness. Although fingerprinting research for LLMs remains in its infancy, insights from smaller-scale fingerprinting efforts suggest a promising outlook. As single-model fingerprinting has advanced, we expect LLM fingerprint recognition to evolve similarly, providing reliable methods of verifying ownership while preserving model performance.

3.1 Fingerprints Based on Vector Direction

Two recent LLM fingerprinting methods hinge on the direction of parameter vectors. Zeng et al. [195] propose a **human-readable fingerprint** (**HuRef**) based on the observation that an LLM's

Table 6. Comparison of Representative Fingerprint Work for IP Protection

Methods/ References		Prepared Model	Test Cases	Fingerprint Comparison	Imperceptibility	Construction Scenarios	Verification Scenarios	Robustness
Direction -based	Zeng et al. [195]	No	Vector direction invariant and visualized image	threshold for directional consistency of a vector	Yes	White-box	Black-box	RLHF/FT
	XU et al. [177]	No	Instructions with secret messages	threshold for the success rate of an instruction with a key	Yes	White-box	Black-box	LoRA/FT
Weight -based	Jia et al. [57]	No	Training samples	Training path recovery	Yes	White-box	White-box	–
	Chen et al. [17]	No	Maximizing activation	threshold for majority voting	Yes	White-box	White-box	FT/FP
	Zheng et al. [209]	No	–	Similarity threshold on random projection	Yes	White-box	White-box	FT/FP
	Chenet al. [16]	No	–	A threshold on hamming distances	Yes	White-box	White-box	FT/FP
	Xue et al. [181]	No	–	Similarity threshold in interpretability information	Yes	White-box	White-box	FT/FP
Behavior -based	Cao et al. [10]	No	Near-boundary samples	A preset threshold on matching rate	No	White-box	Black-box	FT/FP
	Zhou et al. [202]	Yes	Adversarial marks	A threshold on MR from Ghost Networks	No	White-box	Black-box	FT/FP
	Wang et al. [164]	No	Adversarial samples by the deepfool method	A Confidence threshold on Bayesian Probability	No	White-box	Black-box	FT/FP
	Peng et al. [116]	Yes	Universal adversarial perturbations	A fingerprint encoder & Two-Sample t-Test	No	Black-box	Black-box	FT/FP/AT
	Lukas et al. [102]	Yes	Conferrable samples from an ensemble model	A preset threshold on matching rate	Yes	White-box	Black-box	FT/FP /AT/ME
	Li et al. [89]	No	Adversarial sample Pairs to maximize test scores	Data-driven thresholds on CosSim of DDVs	No	White&Black	Black-box	FT/FP/KD /ME/TL
	Yang et al. [184]	Yes	Trigger samples Optimized by contrast loss	A preset threshold on matching rate	No	White-box	Black-box	FT/FP/IM
	Ren et al. [124]	Yes	Gangenerated conferrable samples	A preset threshold on matching rate	Yes	White-box	Black-box	FT/FP /AT/ME
	Pan et al. [112]	Yes	Adaptive fingerprints on model ensemble	Binary classifier network	No	White-box	Black-box	FT/FP
	Guan et al. [41]	SAC-w:Y SAC-c:N	Wrongly classified samples/ CutMix augmented samples	A threshold on metric distances	Yes	White-box	Black-box	FT/FP/AT /ME/TL
	Chen et al. [17]	No	Adversarial samples	threshold for majority voting	No	White-box	Black-box	FT/FP/ME
	Jiang et al. [61]	No	Adversarial samples	A threshold on certified matching rate	No	White-box	Black-box	FT/KD
	Hu et al. [52]	No	Private set sample	A threshold for privacy leakage on a private set	No	White&Black	Black-box	FT/KD/ME

LoRA, RLHF, FT, FP, AT, KD, ME, IM, and TL are some common model post-processing technologies used to verify whether the extracted fingerprints are robust. LoRA and RLHF are unique post-processing operations for large models. FT, FP, AT, KD, ME, IM, and TL represent fine-tuning, micro-pruning, adversarial training, knowledge distillation, model stealing, input modification, and transfer learning.

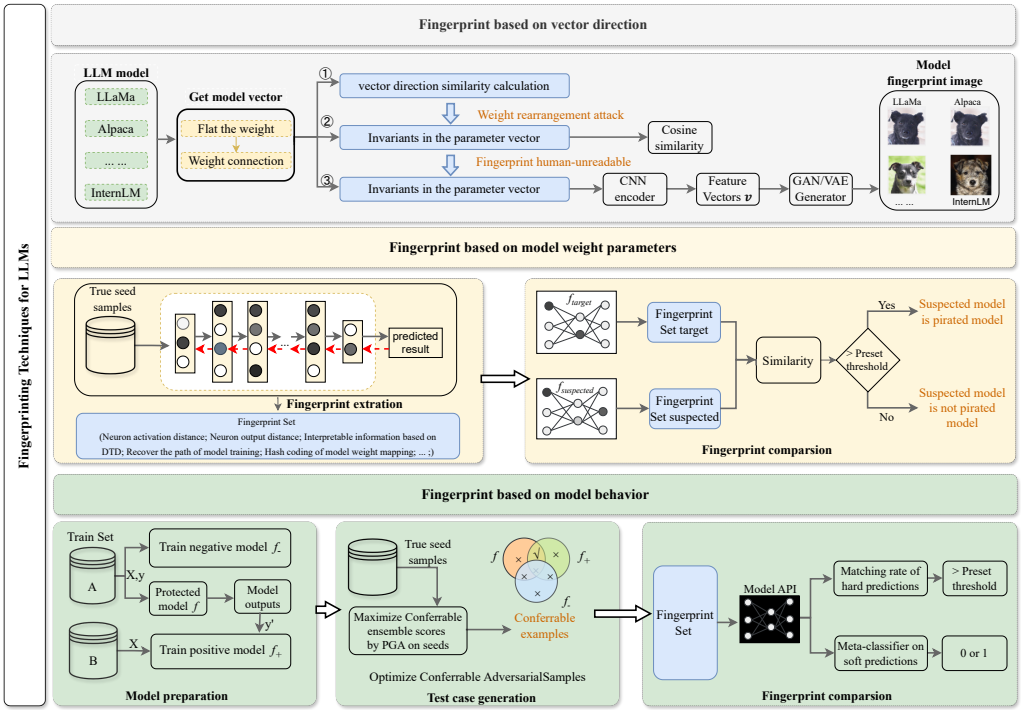


Fig. 2. An illustration of representative research directions in fingerprint-based LLM intellectual property protection.

parameter vector direction remains largely stable upon pre-training convergence. HuRef identifies base models without revealing core parameters or interfering with training. It comprises three main steps:

(1) **Parameter Vector Direction Similarity.** All weights and biases are flattened and concatenated to form a single vector. The cosine similarity,

$$CPD = \frac{|\langle V_A, V_B \rangle|}{\|V_A\| \|V_B\|}, \tag{17}$$

evaluates the consistency of the parameter vector directions between the victim (vector V_A) and suspicious (vector V_B) LLM. A high CPD value indicates potential piracy.

(2) **Invariants in the Parameter Vector.** To prevent full disclosure of the LLM's parameters or easy evasion via structural attacks (e.g., altering W_Q, W_K, W_V, W_O), HuRef defines three *invariants* $(M_a, M_b, M_f) \in \mathbb{R}^{k \times k}$ derived from the LLM's vector direction, which characterize the LLM's unique identity. Cosine similarity of these invariants determines the degree of model similarity.

(3) **Human-Readable Fingerprint Images.** To visualize these invariants, HuRef applies a convolutional encoder to map them into Gaussian variables and then uses StyleGAN2 to produce natural-looking images. Fingerprint validation involves comparing these images across different LLMs.

Although HuRef is effective and does not alter training, it discloses the three invariants and generated fingerprint images. This may risk leaking model insights, as Chen et al. [17] warn about inadvertently exposing model structure, optimization algorithms, or batch sizes from published

plots. Xu et al. [177] introduce another **fingerprinting scheme** (**IF**), implanting a backdoor instruction as the fingerprint. When the model encounters a secret key in the prompt, it produces a predefined response, implying the LLM is pirated. While called a "fingerprint," this method is functionally a backdoor watermark, requiring training-phase modifications. Six quantitative metrics (Harmlessness, Effectiveness, Persistence, Efficiency, Reliability, Robustness) measure its performance, but it fundamentally differs from post-training methods like HuRef.

3.2 Fingerprints Based on Model Weight Parameters

When verifying model ownership with full (white-box) access to suspicious model weights, one can compare models along five dimensions:

(1) **Recovering the Model's Training Path:** PoL [57] is the earliest weight-based fingerprint approach, saving selected data batches and metadata to reconstruct partial training trajectories. Genuine evidence reproduces authentic training fragments, while spurious evidence cannot. This enables directed weight comparisons to prove copyright.
(2) **Measuring Model Weight Distance:** DeepJudge [17] computes multiple distance metrics (neuron output distance, neuron activation distance, layer output distance, layer activation distance) under both white- and black-box settings. Although it measures similarity effectively, distinguishing pirated vs. unrelated LLMs remains challenging.
(3) **Random Projections of Model Weights:** Zheng et al. [209] propose a fingerprinting scheme using **side-channel attacks** (**SCA**) to extract front-layer weights and project them onto a random space defined by an owner's identity. Time-stamping these projections via a trusted third party ensures traceability and irreproducible ownership evidence.
(4) **Learnable Hash Representing Model Similarity:** Inspired by image-retrieval hash methods [25, 155, 212], Chen et al. [16] develop a CNN-based perceptual hashing. Feature distillation selects key features, and normal test statistics produce robust, fixed-length features. Encoding and encrypting these features yield hash codes for efficient Hamming-distance comparisons.
(5) **Interpretability-Based Model Similarity:** InFIP [181] leverages **deep Taylor decomposition** (**DTD**) to extract each model's intrinsic features as a unique "interpretation fingerprint." **Structural similarity** (**SSIM**) [168] between the original and suspicious models' fingerprints determines ownership. A higher SSIM above threshold t indicates piracy.

Although these weight-based approaches can confirm ownership effectively under white-box conditions, they often face limited robustness and practical applicability when access to the suspect model's parameters is restricted, thereby reducing broader scalability.

3.3 Fingerprints Based on Model Behavior

Research on model fingerprints in black-box scenarios typically analyzes a model's unique behavior through test samples, including adversarial examples and samples optimized via stochastic gradient descent. Multiple test samples are combined to form the model's fingerprint. The fingerprinting process involves three main steps: data/model preparation, test case generation, and fingerprint comparison. Below, we describe each step, focusing on key representative works and their application in intellectual property construction and verification.

(1) **Data and Model Preparation:** For accurate IP detection, additional models and datasets are often required to generate test samples and perform fingerprint comparison. Post-processing techniques are used to modify the source model (f), creating a positive (pirated) model (f_+). Examples include CEM [102] and AFA [202], which use ghost networks as positive models. MetaFinger [184] introduces randomness by adding Gaussian noise to model parameters,

while MetaV [112] uses fine-tuning, pruning, and distillation to create multiple positive models. Negative models ($f_$) are trained from scratch on datasets, either identical or differing from the original training set. The training set is divided into non-overlapping portions for source and negative models to reduce false positives. This setup provides a challenging identification scenario.

(2) **Test Sample Generation:** The key focus in model fingerprinting is ensuring the robustness and uniqueness of generated fingerprints. Robustness ensures the fingerprint of the source model persists in modified versions, while uniqueness guarantees that fingerprints distinguish the source model from the negative model. Test samples are selected near the model's classification boundaries to achieve this. Adversarial techniques, such as **Fast Gradient Sign Method** (**FGSM**) [36], **Projected Gradient Descent** (**PGD**) [103], and **Carlini & Wagner** (**C&W**) [11], generate perturbations that alter model behavior and decision labels. Below, we summarize works focused on adversarial example generation and the transferability of these samples for effective fingerprint creation.
   — **Basic Instances:** IPguard [10], one of the first fingerprinting methods, assumes that a source model is uniquely represented by its decision boundary. Using gradient descent, the method optimizes test samples around the source model's decision boundary. This optimization minimizes perturbations that bring test points close to the boundary, balancing robustness and uniqueness. Wang et al. [164] extend this by applying DeepFool [106] to extract boundary points and employing Bayes' theorem for ownership verification. Peng et al. [116] explore **Universal Adversarial Perturbation** (**UAP**) to optimize test samples, leveraging different model decision contours.
   — **Transferability of Test Cases:** IPguard and Deepfool focus on decision boundaries, but these methods often overlap between the source and negative models, making it difficult to verify pirated models under certain adaptive attacks. To address this, CEM [102] synthesizes transferable adversarial samples, integrating positive and negative models for greater robustness. The optimization combines multiple cross-entropy terms and transferability scores to select effective test samples.
   — **Test Scores as Objectives:** Li et al. [89] introduce ModelDiff, which pairs normal and adversarial samples, maximizing test score divergence and diversity to identify unique fingerprints. This method includes a criteria-guided search for generating test samples, overcoming the challenge of missing gradient information in black-box scenarios.
   — **Contrastive Loss:** Traditional methods based on decision boundaries struggle against model modifications and defenses. MetaFinger [184] uses meta-training to generate test examples from the model's internal decision region, shared only by positive models. This approach employs contrastive loss to guide meta-training, optimizing the separation between positive and negative models. GanFinger [124] further improves this by using generative adversarial networks to create realistic test samples, enhancing the robustness of fingerprint verification.
   — **Task-Irrelevant Test Examples:** Many traditional methods rely on classification task-specific features, limiting their general applicability. MetaV [112] introduces a task-independent fingerprinting framework, optimizing adaptive test samples alongside a meta-validator, which improves robustness in diverse tasks.
   — **Sample Relationships Between Test Samples:** SAC [41] considers the relationships between samples as fingerprints, identifying misclassified samples by the source and positive models, but correctly classified by the negative model.
   — **Other Approaches:** Deepjudge [17] offers a solution for both black-box and white-box scenarios, measuring similarity using robustness and Jensen-Shannon distance. IPCert

[61] improves robustness by optimizing stochastic smoothing frameworks for fingerprint verification. VeriDIP [52] uses membership inference to calculate the privacy leakage degree in suspicious models.

(3) **Fingerprint comparison:** After extracting test samples from the source model and suspicious models to construct fingerprints, IP detection needs to be performed by measuring the similarity of the model's behavior patterns on these test samples. The suspicious model is pirated if the similarity is higher than the threshold. Otherwise, it is an irrelevant model. Therefore, setting a threshold has become an important means of evaluating the performance of fingerprints. We divide the existing work into empirically setting thresholds and using deep learning technology to learn thresholds from large amounts of data.

— **Empirically set thresholds for fingerprint comparison:** Most fingerprinting methods [10, 102, 164, 184, 202] are based on comparing the label matching rate of the test sample with a preset threshold to determine whether the suspicious model is a pirated version of the source model. The preset threshold is generally calculated by manually observing many similar model datasets. In particular, the label **matching rate** (MR) only refers to the consistency of the target source model and the suspicious model on the test sample output label, $MR(f_1, f_2) = \frac{|X_m|}{|X_s|}, X_m = \{x \mid f_1(x) = f_2(x), \forall x \in X_s\}$. ModelDiff [89] compares the similarity of models by applying the cosine similarity of the **decision boundary distance vector** (**DDV**), where each element of the DDV is the cosine distance of the model's soft predicted probability for a pair of inputs (x, x').

In SAC [41], the correlation matrix output by the two models on the test sample is calculated through cosine similarity or Gaussian RBF. If the difference in the correlation matrix is higher than the threshold, it is pirated. DeepJudge [112] has designed more comprehensive and fine-grained test indicators at three levels: ① Neuron-level, such as **neuron output distance** (**NOD**) and b (**NAD**); ②Layer-level, such as **layer activation distance** (**LAD**) and **Jensen-Shannon distance** (**JSD**); ③Property-level, such as **robust distance** (**RobD**). Finally, the similarity of the behavioral model is obtained through voting results and weighted summation. Although the above methods of detecting IP by comparison with thresholds are simple, direct, and effective, the empirical threshold design method has higher requirements and is subject to human subjective initiative.

— **Use deep learning models to learn thresholds for fingerprint comparison automatically:** The performance of the model fingerprints is very sensitive to the selection of thresholds. Therefore, a **meta-classifier method** (**MetaV**) [112] is proposed to avoid manually setting thresholds. It concatenates the posterior probabilities of test cases obtained through many positive and negative models into a vector and marks the output vector of the positive model as one and the output vector of the negative model as 0. Finally, a training dataset T is constructed, and a meta-classifier V for binary classification is optimally learned. Therefore, the meta-classifier will automatically optimize and learn an appropriate threshold to classify suspicious models.

3.4 Attacks on Fingerprinting Techniques in LLMs

There are three main types of attacks on fingerprint technology: IP detection, IP evasion, and IP removal. Among them, IP is the abbreviation for intellectual property. IP detection is a process of identifying and detecting whether there are traces of IP protection in a model. IP evasion refers to the behavior that does not trigger or cover up the existing IP protection traces through various technologies. IP removal aims to remove or invalidate IP protection completely from elements in the model [137]. Table 7 summarizes some existing IP attack methods.

Table 7. Summarize the Existing Model Fingerprint Threat Methods

Types	Method	Reference	Data Demands	Data Types	Model Demands	Model Types
IP Detection	Input sample detection	Mika et al. [64]	Poor	–	White-box	–
		Yang et al. [185]	Poor	–	White-box	–
	Model output detection	Sun et al. [146]	Poor	Domain	White-box	–
		Ryoma et al. [136]	Poor	Domain	White-box	–
IP Evasion	Input sample processing	James et al. [48]	Poor	–	White-box	–
		Sahar et al. [1]	Poor	–	White-box	–
		Chen et al. [20]	Poor	–	White-box	–
		Xu et al. [179]	Poor	–	White-box	–
	Model output processing	Kalpesh et al. [73]	Poor	Domain	White-box	-
		Li et al. [82]	Poor	Domain	White-box	–
		Qiang et al. [118]	Poor	Domain	White-box	–
		Shen et al. [139]	Poor	Domain	White-box	–
		Taesung et al. [79]	Poor	Domain	White-box	–
		Zheng et al. [207]	Poor	Domain	White-box	–
IP Removal	Model modification	Kuang et al. [75]	Medium	Labeled/Domain	White-box	Same
		Hu et al. [55]	Medium	Labeled/Domain	White-box	Same
		Lin et al. [95]	Medium	Labeled/Domain	White-box	Same
		Fang et al. [29]	Medium	Domain	White-box	Same
		Wang et al. [167]	Medium	Domain	White-box	Same
		Naman et al. [143]	Medium	Labeled	White-box	Same
		Jia et al. [58]	Medium	Labeled	White-box	Same
		Lukas et al. [37]	Medium	Labeled	White-box	Same
	Model extraction	Aleksandar et al. [70]	Medium	Domain	White-box	Different
		Jiao et al. [62]	Medium	Domain	White-box	Different
		Victor et al. [132]	Medium	Domain	White-box	Different
		Lan et al. [78]	Medium	Domain	White-box	Different
		Sunandini et al. [133]	Strong	Domain	Black-box	Same/Different
		Yue et al. [193]	Strong	Domain	Black-box	Same/Different
		Lewis et al. [9]	Strong	Domain	Black-box	Same/Different
		He et al. [45]	Strong	Domain	Black-box	Same/Different
		Li et al. [90]	Strong	Domain	Black-box	Same/Different

Data demand represents the sample size required by the adversary to avoid IP accountability. Data type indicates whether the data label is a ground-truth label or generated by the source model, where Labeled is the ground-truth and Domain is generated by the source model. Model demand represents the adversary's requirement for knowledge of the source model. Model type indicates whether the pirated model obtained by the adversary is the same as the source model architecture.

3.4.1 IP Detection. According to the different detection objects, IP detection can be divided into input sample and model output detection. IP detection is also utilized to facilitate IP evasion and IP removal [64].

(1) **Input sample detection:** Input sample detection to avoid IP accountability is done by analyzing the distribution differences between IP verification samples and normal samples, relying on comparing the characteristics of the two types of samples. For example, Mika et al. [64] perform a sequential order test to determine whether the distance between the query data before and after a particular user's period roughly follows a Gaussian distribution. If the user's query sample follows a Gaussian distribution, it is considered a benign user, and vice versa.

Yang et al. [185] introduced a method named divergent N-Gram analysis (DNA-GPT). For a given text, they truncate it midway and input only the first part into LLMs to generate the latter sections anew. Through N-gram analysis in probability divergence, they effectively highlight the differences between watermarking text (machine-generated) and clean text (human-written text). By analyzing the characteristics of input samples, input sample detection can adapt to various models and application scenarios, enhancing the universality of the method. However, the complexity and diversity of input samples may limit accuracy.

(2) **Model output detection:** Model output detection is designed to detect whether the output state of the model is abnormal to avoid IP accountability. Because the IP verification samples generated by the fingerprint or backdoor-based watermarking method often rely on inducing the model to output specific errors or anomalies as the basis for IP protection [136]. Since most fingerprint-based IP protection methods rely on adversarial example attacks, adversarial example detection provides an idea for IP detection.

Similarly, backdoor detection technology provides a reference for IP detection based on backdoor watermarking methods. For example, Sun et al. proposed IP detection for machine translation and dialog generation of large language models [146]. Specifically, they perturb the source statement slightly, causing slight or no changes in the source semantics. If the insignificant semantic changes in the source statement result in significant semantic changes in the model output, then the source statement may be an IP verification sample. Model output detection can directly reflect the characteristics and behavior of the model, and by analyzing the output, potential infringement behavior can be intuitively identified. However, real-time monitoring and analysis of many model outputs may require significant computing resources, especially in high-concurrency usage environments. In addition, due to the complexity of the model output, model output detection may encounter false positives (misjudging legitimate use as infringement) or false negatives (failing to detect actual infringement behavior).

3.4.2 IP Evasion. According to the different processing objects, IP evasion can be divided into two camps: input sample processing and model output processing.

(1) **Input sample processing:** Input sample processing refers to manipulating and transforming a sample before it is fed into the model. These operations may include smoothing, feature compression, and so on. The purpose is to damage the quality of IP verification samples and avoid IP accountability.

— **Input noising:** The noise method involves incorporating random fluctuations into a sample, which translates to modifying the words or characters based on values drawn from Gaussian (or normal) distributions in the context of textual data. For example, character-level noise [48] can be introduced by randomly substituting, deleting, inserting, or swapping characters within words. For instance, a character 'e' might be replaced with 'a', or an extra 's' might be inserted, subtly altering the text's appearance while maintaining its overall

readability. Word-level noise [1] can also be applied at the word level by replacing words with synonyms, introducing typographical errors, or shuffling words within a sentence. For example, the word "quick" might be replaced with "fast," or the order of words in the phrase "the brown fox" might be randomly altered. By adding small noise to the model input, the model's decision-making process can be effectively confused, making it difficult for IP verification to accurately determine the source and authenticity of the model, providing a possible means of escaping IP verification. This method is suitable for verifying IP tracking based on model output features. However, the disadvantages of this method are also very obvious. Firstly, input noise may reduce the output quality of the model and affect the user experience. Secondly, excessive noise may cause the model to lose its original functionality and efficiency, damaging its practical value.

— **Input smoothing:** Input smoothing technology requires using a specified kernel (i.e., mathematical matrix) to convolve with the image [103]. This convolution process systematically combines the kernel with image pixels, effectively averaging the pixel values near the kernel size and shape definitions. Therefore, this operation introduces a deliberate blurring effect on the image, making its appearance less clear. The degree of ambiguity is determined by the specific characteristics of the kernel used, such as its size and values within the kernel. Changing these kernel attributes can control the degree and nature of image smoothing. In traditional input smoothing methods, the same number of samples is usually used to establish confidence intervals without considering the specificity of the input data. This **input-insensitive sampling strategy** (**IAS**) may result in an unsatisfactory balance between the **Average Certification radius** (**ACR**) and computational efficiency during model copyright verification, thereby affecting the efficiency of copyright verification.

  To improve the efficiency of the IP verification process, Chen et al. [20] propose an **Input Specific Sampling** (**ISS**) method based on input features, which optimizes the cost-effectiveness of IP verification by dynamically adjusting the sampling quantity. This method adjusts the required sample size based on the characteristics of the input data, improving processing speed. By applying smoothing to input data, the features of the data can be blurred to a certain extent, which helps to mask the unique patterns of model processing and thus helps to evade IP verification based on model output or behavior. This method can also affect the model's output by changing the input data without directly modifying the model itself, increasing the difficulty of IP verification. However, the drawbacks of this method are also very obvious. Input smoothing may lead to the loss of key information in input data, affecting the quality and accuracy of model output and thus damaging the practical value of the model.

— **Feature squeezing:** Feature squeezing technology functions by quantizing the input values, which involves rounding these values to the nearest predetermined quantum level [179]. This process effectively compresses the range of input data by reducing its precision. Xu et al. [179] focus their research on two primary types of compression techniques: reducing the color depth of images and applying smoothing techniques, both local and non-local, to minimize variations between pixels.

  Reducing color depth involves decreasing the number of bits used to represent each pixel's color in the image. On the other hand, smoothing techniques are employed to reduce noise and variations between adjacent pixels. Local smoothing typically involves filters, such as Gaussian or median filters, which consider the values of a pixel's immediate neighbors to adjust its value, smoothing out rapid intensity changes and noise. Non-local smoothing extends this concept by also considering the similarity between distant pixels or regions within the image, allowing for more effective noise reduction and preservation

of significant features by selectively averaging pixels based on their similarity, regardless of their spatial proximity. These compression strategies are particularly useful when bandwidth or storage is limited or computational efficiency is crucial. However, the extent to which these technologies are applied must be carefully balanced with the need to preserve important image details and quality.

(2) **Model output processing:** Model output processing refers to manipulating and transforming a model's output information. This includes dealing with perturbing, replacing, and so on, to ensure that the effectiveness of IP protection in the output information is maintained.

— **Paraphrase attacks:** Paraphrasing exploits the complexity of natural language and the limitations of the model's understanding of linguistic nuances [104]. By modifying the text, the adversary can create a new version that confuses IP verification without changing the general meaning of the text [144]. For example, Kalpesh et al. [73] propose a paraphrasing attack. Firstly, they trained a paraphrasing generation model with 11 billion parameters (called DIPPER) capable of paraphrasing paragraphs, selectively using surrounding text (for example, user-written prompts) as context. DIPPER also uses scalar knobs to control the diversity of vocabulary and the degree of reordering in definitions. Li et al. [82] propose a paraphrasing-based attack method to target summarization models.

Initially, they rank the sentences within a document based on their influence on the summary. Subsequently, they employ a paraphrasing procedure to generate adversarial examples. While the adversarial examples have a high semantic similarity with the original sentence, the integrity of the original sentence is damaged, weakening the performance of IP protection. Qiang et al. [118] propose paraphraser-based lexical substitution. Initially, they leverage a paraphrase dataset to train a neural network-based paraphraser. Following this, after inputting sentence x into the paraphraser, they utilize a specialized decoding technology to extract potential substitutes for the target word. Finally, they rank these substitute candidates, selecting the most suitable option that preserves the original meaning of sentence x without alteration. Paraphrase attacks can be very subtle, making it difficult for IP protection methods to detect them since the paraphrased content can retain the same meaning as the original.

Paraphrase attacks can be tailored to specific contexts or content types, making them versatile tools for circumventing IP protections across various domains and languages. However, generating high-quality paraphrases that maintain the original meaning while being sufficiently different to avoid detection can be technically challenging, requiring state-of-the-art NLP technologies and computational resources.

— **Perturbation:** Perturbation refers to a method of perturbing the output generated by the model, designed to bypass or interfere with IP verification [163]. For text-generative models, this method may involve changing some of the vocabulary, sentence structure, or semantic details of the model's output to produce something similar to, but not identical to, the original output. Compared to paraphrase attacks, the perturbation method does not require training in paraphrasing. It only needs to select the vocabulary closest to the replacement word in the existing dictionary. For example, Shen et al. [139] utilize **term frequency-inverse document frequency** (**TF-IDF**) [130], a metric that measures the importance of a word within a collection or corpus of documents, to identify significant words. Following the injection of perturbations using $\eta d\chi$-privacy method [31], they then search for the nearest token to serve as a replacement. This method may involve perturbing the model's output posterior probability distribution, generating a posterior probability different from the original posterior probability distribution, but not affecting the final classification result for text classification or sentiment analysis models.

Taesung et al. [79] craft a perturbation technology tailored for classification models, specifically designed for scenarios where posterior probabilities are fed back to the user. Zheng et al. [207] introduce **boundary differential privacy** (**BDP**) as a perturbation technology. The primary goal of BDP is to obfuscate the model's decision boundaries. When an adversary's query samples fall within the decision boundary region, BDP provides obfuscated posterior probability predictions. The work of Zheng et al. [207] is based on the observation that most of the fingerprint samples of the model are located near the decision boundary. Perturbation methods are relatively straightforward, requiring minimal changes to the model output, making them accessible even to those with limited technical expertise. However, excessive or poorly designed perturbations can degrade the quality of the model's outputs, leading to reduced utility or credibility of the generated content.

3.4.3 IP Removal. Depending on the amount of knowledge required, IP removal can be classified into two categories: Model Modification and Model Extraction.

(1) **Model modification:** Model modification requires a detailed understanding of the internal working mechanism of the model, which is called white-box access. In this case, the adversary can directly access the parameters and architecture of the model. He makes modifications to remove or modify the IP protection embedded in the model, such as watermarks or specific identifiers.
   — **Fine-Tuning:** Fine-tuning involves fine-tuning and training of parameters based on pre-trained models (illegally copied pirated models), enabling the model to better adapt to new specific tasks or datasets [95]. This process often only involves modifying a small portion of the model's layers or parameters while keeping most of the pre-trained architecture unchanged [55]. Although this allows the fine-tuned model to demonstrate high performance on new tasks, similarly, such subtle adjustments may also remove the difficulty of IP verification mechanisms in distinguishing the differences between the fine-tuned model and its original pre-trained model, thereby reducing the accuracy of IP verification [75].

   There are four main types of fine-tuning methods: **Fine-Tune All Layers** (**FTAL**), **Fine-Tune Last Layer** (**FTLL**), **Retrain All Layers** (**RTAL**), and **Retrain Last Layer** (**RTLL**). FTAL refers to all the weights of the source model undergoing fine-tuning, which means that each parameter within the model is adjusted and optimized based on new data. FTLL refers to the last layer of the source model being fine-tuned and all other weights being frozen. RTAL refers to reinitializing the weights of the last layer of the source model and fine-tuning all weights. RTLL refers to the reinitialization of the weights of the last layer of the source model, with only the weights of the last layer being fine-tuned. By making subtle adjustments to a pre-trained model, it is possible to effectively alter the model's output without significantly changing the overall architecture, potentially circumventing IP verification mechanisms that are based on output characteristics. This method involves modifying certain layers or parameters of the model to adapt it to new tasks or datasets, which might also obscure the model's origin from IP verification mechanisms. However, the drawback of this method lies in its potential impact on model performance, especially when the fine-tuning is not conducted properly, or there is insufficient data to support effective learning.
   — **Pruning:** Pruning is a technique that narrows the volume of a neural network by removing parts of the model that contribute the least to its output predictions [167]. This can involve eliminating weights, neurons, or even entire layers deemed less important based on specific criteria, such as those with the smallest magnitude weights or those that have the least impact on model performance [46]. For example, Fang et al. [29] introduce DepGraph,

a universally automated approach that systematically identifies layer dependencies and groups parameters for effective pruning across diverse models. This method overcomes the challenge of architecture-specific pruning tools, which rely on manually crafted grouping schemes and often fail to generalize across new network architectures. DepGraph is designed to address the complex task of structural pruning for various architectures, such as CNNs, RNNs, GNNs, and Transformers, by managing structural couplings and ensuring the consistent removal of non-essential parameters.

Pruning, by removing redundant or insignificant parameters and neurons, can alter the internal structure of a model, potentially affecting the accuracy of IP verification. Such structural changes can obscure copyright detection algorithms, making it challenging to identify the model's source. However, pruning may reduce the model's performance, especially if over-pruned, leading to the loss of important information for the original task. Moreover, the pruning process must be conducted cautiously to avoid fundamental damage to the model's functionality.

— **Adversarial training:** Adversarial training is a method to improve the robustness of machine learning models [6]. The general process of adversarial training includes periodically generating adversarial examples during the training process and incorporating these examples into the training set [58]. Specifically, the current model is first used to identify or generate perturbation inputs (adversarial samples) that can maximize model error, and then these adversarial examples are used together with the original training data for training, forcing the model to learn to maintain stable predictions when facing potential perturbations [37]. For example, Naman et al. [143] focus on the adversarial training of Visual Converters (ViTs) ConvNeXt. The key aspects of their method include architectural modifications, such as replacing PatchStem with ConvStem, and refinements in the training scheme.

The primary advantage of adversarial training is its ability to introduce subtle changes to the model's decision-making process, potentially rendering copyright verification based on model behavior less reliable. By exposing the model to adversarially modified inputs during training, the model learns to generalize better and might develop characteristics that deviate from those of the original pre-trained model, complicating the task of copyright attribution. However, the main drawback is the risk of degrading the model's performance on legitimate tasks.

(2) **Model extraction:** Model extraction does not require in-depth knowledge of the details inside the model but operates through the model's input and output, known as black-box access. In model extraction, by observing the behavior and response of the model, the adversary attempts to train a new model that is functionally similar to the source model but does not contain the IP protection features of the source model.

— **Distillation:** Distillation is a technology where knowledge from a complex, typically larger and more sophisticated "teacher" model is transferred to a simpler, smaller "student" model [70]. The process involves training the student model to mimic the behavior of the teacher model, not just by matching the final output predictions but often by aligning with soft probabilities or intermediate representations generated by the teacher [62]. The process from complex to simple model is conducive to removing traces of IP verification. For example, DistilBERT [132] enhances efficiency by streamlining BERT's architecture, notably halving the Transformer layers to reduce complexity and computational needs.

DistilBERT also utilizes a multi-task learning approach that integrates language modeling with other potential task-specific objectives and employs optimization and regularization technologies such as dynamic learning rate adjustment and weight decay to improve

generalization and efficiency. ALBERT [78] introduced by Google, is an optimization of the original **Bidirectional Encoder Representations from Transformers** (**BERT**) model, designed to improve efficiency and reduce model size without significantly sacrificing performance. ALBERT implements two main innovations: parameter-sharing across all layers, which reduces the model's memory footprint, and the decomposition of the embedding matrix into two smaller matrices, effectively decreasing the parameters associated with word embeddings. Distillation allows the creation of a new model that captures the essential functionality of the original without duplicating its exact architecture or parameters, potentially removing IP traces. However, the primary drawback is that the distilled model may not achieve the same level of performance as the original, given that some nuances and complexities of the original model might not be fully transferred.

— **Model extraction attacks:** Model extraction attacks aim at replicating the functionality of a proprietary or black-box model [133]. This is achieved by systematically querying the target model with carefully crafted inputs and analyzing the corresponding outputs [193]. The goal is to infer and reconstruct a model that closely mimics the behavior and decision-making process of the source without having direct access to its internal architecture, parameters, or training data [9]. This method enables the adversary to create a functionally similar model, which can then be used independently, posing potential threats to the intellectual property and competitive advantages of the source model's creators. For example, He et al. [45] propose DRMI, a method for model extraction attacks using mutual information technology. DRMI uses an optimization algorithm that is designed to pick out a subset of samples that are rich in information. The mutual information of these subsets is minimized to steal surrogate models with decision behavior similar to the target model while reducing the frequency of queries to the target model. Li et al. [90] have made breakthrough progress in large models that convert natural language into executable code tasks, successfully replicating pirated models that match the performance of the source model at lower query costs. Model extraction attacks can create a functionally similar model without direct access to the original model's architecture or parameters, potentially evading detection by IP verification. However, model extraction attacks can lead to a degradation in the performance of the extracted model compared to the original due to the imperfect replication process.

4 Current Challenges and Future Directions

The application of watermarking and fingerprinting techniques in LLMs offers considerable potential but presents numerous challenges. The journey toward effective implementation is laden with complexities, which arise from the characteristics of LLMs and the evolving digital security landscape.

4.1 Watermarking Techniques in LLMs

One of the hurdles of watermarking techniques is scalability and efficiency. Embedding a watermark without compromising the model's performance or incurring substantial computational overhead is a difficult balance to strike. Moreover, the robustness of watermarks against adversarial attacks is a major concern. Attackers may employ various strategies, such as fine-tuning the model on a new dataset, pruning parameters to reduce model size, or using knowledge distillation to transfer knowledge to a smaller, watermark-free model. These attacks can potentially remove or alter the embedded watermark. Another challenge is the detectability and extraction of the watermark. The watermark must be designed to be reliably detected under different conditions to prove ownership.

However, this requirement must be balanced with the need to keep the watermark hidden from potential attackers, who might attempt to remove it if detected.

Recent studies on open-source LLM checkpoints reveal that sampling- and logits-based watermarks can usually be detected with high confidence when the model is queried in its original language and domain, yet their resilience drops once real-world transformations are applied. For example, lightweight domain adaptation to customer-service dialogue, a single epoch of parameter-efficient fine-tuning on proprietary data, or translation reduces the statistical margin between watermarked and clean text to the point where detector thresholds must be relaxed to avoid false negatives. Controlled experiments further show that subtle pruning of less-attended heads or low-magnitude weight rows can suppress watermark signals without measurable impact on task accuracy, while knowledge-distilled student models inherit less than half of the parent watermark's footprint.

Legal and ethical considerations also play a role in implementing watermarking techniques in LLMs. Issues such as user privacy, data ownership, and the potential misuse of watermarked models need to be addressed. For instance, watermarking methods that modify the training data or model outputs might raise privacy concerns if sensitive information is embedded in the watermark. Similarly, the ownership of the watermark and the model itself can become contentious, especially in collaborative environments where multiple parties contribute to the model's development. Beyond these considerations lies the risk of unjust attribution. As watermarking and detection tools become ubiquitous, false positives or adversarially crafted "planted" watermarks could lead to erroneous accusations of model theft or output plagiarism. Such misuse threatens both scientific integrity and legal fairness. To mitigate these risks, several safeguards have been proposed: (i) adopting statistically rigorous detection thresholds with publicly disclosed error-rate targets; (ii) requiring corroboration from multiple independent detectors or verification by the model owner through cryptographic proofs (e.g., zero-knowledge attestations of secret keys); (iii) maintaining version-controlled audit logs that record the watermarking parameters at the time of model release; and (iv) establishing neutral arbitration panels or standardized dispute-resolution protocols within the research community. Embedding metadata that signals the intended provenance of a model, coupled with transparent benchmarking of detector accuracy on shared corpora, further reduces the scope for malicious or negligent misapplication.

Looking forward, the trajectory of watermarking techniques within LLMs is poised to be shaped by the rapid integration of AI with emerging technologies, the advancement of computational linguistics, and the intensifying legal frameworks surrounding digital rights management. One promising avenue is the exploration of advanced embedding techniques that leverage the inherent properties of LLMs. For instance, researchers may investigate methods that embed watermarks in the attention mechanisms or the latent representations of LLMs, offering a more robust and imperceptible way to embed watermarks. Such techniques could make the watermark more resilient to adversarial attacks and model modifications. Another key area of focus will be the development of dynamic watermarking schemes that can adapt to the changing nature of LLMs. Static watermarks may become outdated or less effective as LLMs are updated with new data and fine-tuned for specific tasks. Dynamic watermarking approaches that can update the watermark in response to model changes could ensure continuous IP protection. Furthermore, integrating watermarking with other security measures, such as encryption and access control mechanisms, could provide a multi-layered defense against unauthorized use. By combining different security techniques, the overall resilience of LLMs to various types of attacks can be enhanced. More importantly, developing standardized protocols and benchmarks for evaluating the effectiveness of watermarking techniques in LLMs is valuable. Establishing common standards and metrics will enable the comparison of different watermarking approaches. Benchmarks that simulate real-world attack scenarios can

provide valuable insights into the resilience of watermarking techniques, guiding the design of more robust solutions.

4.2 Fingerprinting Techniques in LLMs

One of the challenges is the extraction of reliable and distinctive fingerprints from LLMs. Due to these models' high-dimensional nature, identifying features that can serve as robust fingerprints is difficult. Unlike watermarking, where additional information is embedded into the model, fingerprinting relies on inherent characteristics of the model, which may be hard to isolate. Another challenge is the variability of LLMs. These models are often fine-tuned or adapted for specific tasks, resulting in variations of the same base model. The process of fine-tuning or adaptation itself may inadvertently alter or obscure the fingerprints, necessitating the development of resilient approaches to such modifications. Given the vast parameter space of LLMs, efficiently querying the model to extract its fingerprint without extensive computational resources is daunting. Thus, the detection process must be sensitive enough to identify the unique fingerprint of an LLM while being robust enough to avoid false positives that may arise from the natural variability of the model's outputs. In addition to these technical challenges, some practical considerations are related to deploying fingerprinting techniques in real-world scenarios. Ensuring that the fingerprinting process is scalable, efficient, and can be integrated into existing workflows is essential for widespread adoption.

Evaluations of fingerprinting methods indicate that feature-direction and behaviour-based fingerprints remain identifiable under benign fine-tuning. However, their reliability degrades once models undergo aggressive compression or cross-modal alignment. In code-generation and multimodal tasks, for instance, sentence-level querying patterns extracted from the base model are often blurred after adapters or visual encoders are grafted onto the backbone, requiring ensemble detectors that aggregate multiple weak signals. Reports show that fingerprint match rates stay stable when clients adjust temperature or decoding length, yet drop sharply when responses are rephrased by external paraphrase engines or routed through machine-translation pipelines. Moreover, federated settings in which several derivative models are averaged or merged can erase distinctive parameter-space signatures unless the original fingerprint is reinforced with cryptographically bound noise.

The future directions for fingerprinting techniques in LLMs are poised to address the existing challenges and leverage new opportunities presented by technological advancements. Developing more robust and adaptive fingerprinting methods is promising. These methods should be capable of withstanding some model modifications, such as fine-tuning, pruning, and distillation, without losing effectiveness. Technologies that can adjust to changes in model architecture or parameters will be sought. This may involve exploring novel embedding techniques that integrate with the model's structure or leveraging advanced cryptographic approaches to secure fingerprints against tampering. Secondly, enhancing the scalability of fingerprinting techniques will be valuable. As LLMs continue to grow in size and complexity, fingerprinting methods must be able to operate without imposing significant computational overhead. This involves developing lightweight algorithms that require minimal resources for embedding and verification. Thirdly, diversifying fingerprinting approaches will be essential to accommodate the wide range of LLM applications. Additionally, exploring multimodal fingerprinting strategies that can protect LLMs used in conjunction with other data types, such as images or audio, will be important for comprehensive security solutions. Fourthly, addressing the challenges posed by adversarial attacks will remain a focus. Developing more sophisticated detection mechanisms to counteract attempts to evade or remove fingerprints will be necessary. As research and innovation continue in this field, we can anticipate the emergence of more sophisticated and effective fingerprinting solutions to protect LLMs.

5 Conclusions

In this survey, we have explored the state-of-the-art watermarking and fingerprinting techniques for **Large Language Models** (**LLMs**). Our analysis has spanned a wide array of methodologies, each designed to protect the intellectual property of LLMs and ensure their secure deployment in some applications. Investigating watermarking techniques based on embedding watermarks during the training phase, the logits generation phase, and the token sampling phase. Meanwhile, we also review the application of watermarking technology in LLMs and the potential attacks. Our review highlighted some innovative methods to embed watermarks in LLMs. Moreover, our examination of fingerprinting techniques revealed the ingenuity behind methods used to identify LLMs. We discussed the development of fingerprints based on model behavior, such as responses to adversarial examples, and using deep learning models to learn thresholds for fingerprint comparison. Our survey has underscored the importance of advancing security measures for LLMs, especially in light of the increasing sophistication of adversarial attacks. As LLMs continue to play a pivotal role in advancing AI technologies, developing and refining security measures that safeguard their intellectual property and ensure their ethical deployment is imperative.

References

[1] Sahar Abdelnabi and Mario Fritz. 2021. Adversarial watermarking transformer: Towards tracing text provenance with data hiding. In *Proceedings of the 2021 IEEE Symposium on Security and Privacy*. IEEE, 121–140.

[2] Yelaman Abdullin, Diego Molla, Bahadorreza Ofoghi, et al. 2023. Synthetic dialogue dataset generation using LLM agents. In *Proceedings of the 3rd Workshop on Natural Language Generation, Evaluation, and Metrics*. 181–191.

[3] P. Aberna and L. Agilandeeswari. 2024. Digital image and video watermarking: methodologies, attacks, applications, and future directions. *Multimedia Tools and Applications* 83, 2 (2024), 5531–5591.

[4] Josh Achiam, Steven Adler, Sandhini Agarwal, et al. 2023. GPT-4 Technical Report. arXiv:2303.08774. Retrieved from https://arxiv.org/abs/2303.08774

[5] Anirudh Ajith, Sameer Singh, and Danish Pruthi. 2023. Performance trade-offs of watermarking large language models. arXiv:2311.09816. Retrieved from https://arxiv.org/abs/2311.09816

[6] Maksym Andriushchenko and Nicolas Flammarion. 2020. Understanding and improving fast adversarial training. In *Proceedings of the Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems*.

[7] Michiel Bakker, Martin Chadwick, Hannah Sheahan, et al. 2022. Fine-tuning language models to find agreement among humans with diverse preferences. In *Proceedings of the Advances in Neural Information Processing Systems*. S. Koyejo, S. Mohamed, A. Agarwal, et al. (Eds.), Vol. 35, 38176–38189.

[8] Folco Bertini Baldassini, Huy H. Nguyen, Ching-Chung Chang, et al. 2024. Cross-attention watermarking of large language models. In *Proceedings of the ICASSP 2024-2024 IEEE International Conference on Acoustics, Speech and Signal Processing*. IEEE, 4625–4629.

[9] Lewis Birch, William Hackett, Stefan Trawicki, et al. 2023. Model leeching: An extraction attack targeting llms. arXiv:2309.10544. Retrieved from https://arxiv.org/abs/2309.10544

[10] Xiaoyu Cao, Jinyuan Jia, and Neil Zhenqiang Gong. 2021. IPGuard: Protecting intellectual property of deep neural networks via fingerprinting the classification boundary. In *Proceedings of the 2021 ACM Asia Conference on Computer and Communications Security*. 14–25.

[11] Nicholas Carlini and David Wagner. 2017. Towards evaluating the robustness of neural networks. In *Proceedings of the 2017 IEEE Symposium on Security and Privacy*.

[12] Marco Cascella, Jonathan Montomoli, Valentina Bellini, et al. 2023. Evaluating the feasibility of ChatGPT in healthcare: an analysis of multiple clinical and research scenarios. *Journal of Medical Systems* 47, 1 (2023), 33.

[13] Junbum Cha, Wooyoung Kang, Jonghwan Mun, et al. 2024. Honeybee: Locality-enhanced projector for multimodal llm. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 13817–13827.

[14] Shreyas Chaudhari, Pranjal Aggarwal, Vishvak Murahari, et al. 2024. Rlhf deciphered: A critical analysis of reinforcement learning from human feedback for llms. *Computing Surveys* (2024).

[15] Dongping Chen, Ruoxi Chen, Shilin Zhang, et al. 2024. Mllm-as-a-judge: Assessing multimodal llm-as-a-judge with vision-language benchmark. In *Proceedings of the 41st International Conference on Machine Learning*.

[16] Haozhe Chen, Hang Zhou, Jie Zhang, et al. 2023. Perceptual hashing of deep convolutional neural networks for model copy detection. *ACM Transactions on Multimedia Computing, Communications and Applications* 19, 3 (2023), 1–20.

[17] Jialuo Chen, Jingyi Wang, Tinglan Peng, et al. 2022. Copy, right? a testing framework for copyright protection of deep learning models. In *Proceedings of the 2022 IEEE Symposium on Security and Privacy*. IEEE, 824–841.
[18] Keqin Chen, Zhao Zhang, Weili Zeng, et al. 2023. Shikra: Unleashing multimodal LLM's referential dialogue magic. arXiv:2306.15195. Retrieved from https://arxiv.org/abs/2306.15195
[19] Liang Chen, Yatao Bian, Yang Deng, et al. 2023. X-mark: Towards lossless watermarking through lexical redundancy. arXiv:2311.09832. Retrieved from https://arxiv.org/abs/2311.09832
[20] Ruoxin Chen, Jie Li, Junchi Yan, et al. 2022. Input-specific robustness certification for randomized smoothing. In *Proceedings of the AAAI Conference on Artificial Intelligence*. 6295–6303.
[21] Miranda Christ and Sam Gunn. 2024. Pseudorandom error-correcting codes. In *Proceedings of the Annual International Cryptology Conference*. Springer, 325–347.
[22] Miranda Christ, Sam Gunn, and Or Zamir. 2024. Undetectable watermarks for language models. In *Proceedings of the 37th Annual Conference on Learning Theory*. PMLR, 1125–1139.
[23] Tianyu Cui, Yanling Wang, Chuanpu Fu, et al. 2024. Risk taxonomy, mitigation, and assessment benchmarks of large language model systems. arXiv:2401.05778. Retrieved from https://arxiv.org/abs/2401.05778
[24] Yaodong Cui, Shucheng Huang, Jiaming Zhong, et al. 2023. DriveLLM: Charting the path toward full autonomous driving with large language models. *IEEE Transactions on Intelligent Vehicles* (2023).
[25] Thanh-Toan Do, Tuan Hoang, Dang-Khoa Le Tan, et al. 2019. From selective deep convolutional features to compact binary representations for image retrieval. *ACM Transactions on Multimedia Computing, Communications, and Applications* 15, 2 (2019), 1–22.
[26] Danny Driess, Fei Xia, Mehdi S. M. Sajjadi, et al. 2023. Palm-e: An embodied multimodal language model. arXiv:2303.03378. Retrieved from https://arxiv.org/abs/2303.03378
[27] Jonathan Evertz, Merlin Chlosta, Lea Schönherr, et al. 2024. Whispers in the Machine: Confidentiality in LLM-integrated Systems. arXiv:2402.06922. Retrieved from https://arxiv.org/abs/2402.06922
[28] Jaiden Fairoze, Sanjam Garg, Somesh Jha, et al. 2025. Publicly-detectable watermarking for language models. *IACR Communications in Cryptology* 1, 4 (2025).
[29] Gongfan Fang, Xinyin Ma, Mingli Song, et al. 2023. Depgraph: Towards any structural pruning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 16091–16101.
[30] Pierre Fernandez, Antoine Chaffin, Karim Tit, et al. 2023. Three bricks to consolidate watermarks for large language models. In *Proceedings of the 2023 IEEE International Workshop on Information Forensics and Security*. IEEE, 1–6.
[31] Oluwaseyi Feyisetan, Borja Balle, Thomas Drake, et al. 2020. Privacy-and utility-preserving textual analysis via calibrated multivariate perturbations. In *Proceedings of the 13th International Conference on Web Search and Data Mining*. 178–186.
[32] Jiayi Fu, Xuandong Zhao, Ruihan Yang, et al. 2024. GumbelSoft: Diversified language model watermarking via the gumbelmax-trick. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 5791–5808.
[33] Jorge Gabín and Javier Parapar. 2025. Leveraging retrieval-augmented generation for keyphrase synonym suggestion. In *Proceedings of the European Conference on Information Retrieval*. Springer, 311–327.
[34] Payal Garg and Ajit Jain. 2023. A robust technique for biometric image authentication using invisible watermarking. *Multimedia Tools and Applications* 82, 2 (2023), 2237–2253.
[35] Lea Goetz, Markus Trengove, Artem Trotsyuk, et al. 2023. Unreliable LLM bioethics assistants: Ethical and pedagogical risks. *The American Journal of Bioethics* 23, 10 (2023), 89–91.
[36] Ian J. Goodfellow, Jonathon Shlens, and Christian Szegedy. 2014. Explaining and harnessing adversarial examples. arXiv:1412.6572. Retrieved from https://arxiv.org/abs/1412.6572
[37] Lukas Gosch, Simon Geisler, Daniel Sturm, et al. 2023. Adversarial training for graph neural networks: Pitfalls, solutions, and new directions. *Advances in Neural Information Processing Systems* 36 (2023), 58088–58112.
[38] Przemyslaw Grabowicz, Nicholas Perello, and Yair Zick. 2023. Towards an AI Accountability Policy. arXiv:2307.13658. Retrieved from https://arxiv.org/abs/2307.13658
[39] Chenchen Gu, Xiang Lisa Li, Percy Liang, et al. 2023. On the learnability of watermarks for language models. arXiv:1409.0473. Retrieved from https://arxiv.org/abs/1701.00133
[40] Xiangming Gu, Xiaosen Zheng, Tianyu Pang, et al. 2024. Agent Smith: A single image can jailbreak one million multimodal LLM agents exponentially fast. In *Proceedings of the 41st International Conference on Machine Learning*. 16647–16672.
[41] Jiyang Guan, Jian Liang, and Ran He. 2022. Are you stealing my model? sample correlation for fingerprinting deep neural networks. *NeurIPS2022* (2022).
[42] Jiaxian Guo, Junnan Li, Dongxu Li, et al. 2023. From images to textual prompts: Zero-shot visual question answering with frozen large language models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 10867–10877.

[43] Yucheng Han, Chi Zhang, Xin Chen, et al. 2023. Chartllama: A multimodal llm for chart understanding and generation. arXiv:2311.16483. Retrieved from https://arxiv.org/abs/2311.16483
[44] Pengfei He, Han Xu, Yue Xing, et al. 2025. Data poisoning for in-context learning. In *Findings of the Association for Computational Linguistics: NAACL 2025*. 1680–1700.
[45] Yingzhe He, Guozhu Meng, Kai Chen, et al. 2021. {DRMI}: A dataset reduction technology based on mutual information for black-box attacks. In *Proceedings of the 30th USENIX Security Symposium*. 1901–1918.
[46] Yang He and Lingao Xiao. 2023. Structured pruning for deep convolutional neural networks: A survey. *IEEE Transactions on Pattern Analysis and Machine Intelligence* (2023).
[47] Zhiwei He, Binglin Zhou, Hongkun Hao, et al. 2024. Can watermarks survive translation? on the cross-lingual consistency of text watermark for large language models. In *Proceedings of the ACL*.
[48] James N. Helfrich and Rick Neff. 2012. Dual canonicalization: An answer to the homograph attack. In *Proceedings of the 2012 eCrime Researchers Summit*. IEEE, 1–10.
[49] Abe Hou, Jingyu Zhang, Tianxing He, et al. 2024. SemStamp: A semantic watermark with paraphrastic robustness for text generation. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. 4067–4082.
[50] Chiou-Ting Hsu and Ja-Ling Wu. 1999. Hidden digital watermarks in images. *IEEE Transactions on Image Processing* 8, 1 (1999), 58–68.
[51] Ling-Yuan Hsu, Hwai-Tsu Hu, and Hsien-Hsin Chou. 2022. A high-capacity QRD-based blind color image watermarking algorithm incorporated with AI technologies. *Expert Systems with Applications* 199 (2022), 117134.
[52] Aoting Hu, Zhigang Lu, Renjie Xie, et al. 2023. VeriDIP: Verifying ownership of deep neural networks through privacy leakage fingerprints. *IEEE Transactions on Dependable and Secure Computing* 21, 4 (2023), 2568–2584.
[53] Wenbo Hu, Yifan Xu, Yi Li, et al. 2024. Bliva: A simple multimodal llm for better handling of text-rich visual questions. In *Proceedings of the AAAI Conference on Artificial Intelligence*. 2256–2264.
[54] Zhengmian Hu, Lichang Chen, Xidong Wu, et al. 2023. Unbiased watermark for large language models. arXiv:2310.10669. Retrieved from https://arxiv.org/abs/2310.10669
[55] Zhiqiang Hu, Lei Wang, Yihuai Lan, et al. 2023. LLM-adapters: An adapter family for parameter-efficient fine-tuning of large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 5254–5276.
[56] Baihe Huang, Banghua Zhu, Hanlin Zhu, et al. 2023. Towards optimal statistical watermarking. arXiv:2312.07930. Retrieved from https://arxiv.org/abs/2312.07930
[57] Hengrui Jia, Mohammad Yaghini, Christopher A. Choquette-Choo, et al. 2021. Proof-of-learning: Definitions and practice. In *Proceedings of the 2021 IEEE Symposium on Security and Privacy*. IEEE, 1039–1056.
[58] Xiaojun Jia, Yong Zhang, Baoyuan Wu, et al. 2022. LAS-AT: Adversarial training with learnable attack strategy. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. IEEE, 13388–13398.
[59] Feibo Jiang, Li Dong, Yubo Peng, et al. 2024. Large AI model empowered multimodal semantic communications. *IEEE Communications Magazine* (2024).
[60] Liming Jiang. 2024. Utilizing Large LanguageModels to Detect Privacy Leaks in Mini-App Code. arXiv:2402.07367. Retrieved from https://arxiv.org/abs/2402.07367
[61] Zhengyuan Jiang, Minghong Fang, and Neil Zhenqiang Gong. 2023. Ipcert: Provably robust intellectual property protection for machine learning. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 3612–3621.
[62] Xiaoqi Jiao, Yichun Yin, Lifeng Shang, et al. 2020. TinyBERT: Distilling BERT for natural language understanding. In *Findings of the Association for Computational Linguistics: EMNLP 2020*. Association for Computational Linguistics, 4163–4174.
[63] Yang Jin, Zhicheng Sun, Kun Xu, et al. 2024. Video-LaVIT: Unified video-language pre-training with decoupled visual-motional tokenization. In *Proceedings of the International Conference on Machine Learning*. PMLR, 22185–22209.
[64] Mika Juuti, Sebastian Szyller, Samuel Marchal, et al. 2019. PRADA: Protecting against DNN model stealing attacks. In *Proceedings of the 2019 IEEE European Symposium on Security and Privacy*. IEEE, 512–527.
[65] Asra Kamili, Nasir N. Hurrah, Shabir A. Parah, et al. 2020. DWFCAT: Dual watermarking framework for industrial image authentication and tamper localization. *IEEE Transactions on Industrial Informatics* 17, 7 (2020), 5108–5117.
[66] Nikhil Kandpal, Krishna Pillutla, Alina Oprea, et al. 2023. User inference attacks on LLMs. In *Proceedings of the Socially Responsible Language Modelling Research*.
[67] Kaan Efe Keleş, Ömer Kaan Gürbüz, and Mucahid Kutlu. 2023. I know you did not write that! a sampling based watermarking method for identifying machine generated text. arXiv:2311.18054. Retrieved from https://arxiv.org/abs/2311.18054
[68] Jungin Kim, Shinwoo Park, and Yo-Sub Han. 2025. Marking code without breaking it: Code watermarking for detecting llm-generated code. arXiv:2502.18851. Retrieved from https://arxiv.org/abs/2502.18851

[69] John Kirchenbauer, Jonas Geiping, Yuxin Wen, et al. 2023. A watermark for large language models. In *Proceedings of the International Conference on Machine Learning*. PMLR, 17061–17084.
[70] Aleksandar Kitanovski, Martina Toshevska, and Georgina Mirceva. 2023. Distilbert and roberta models for identification of fake news. In *Proceedings of the 2023 46th MIPRO ICT and Electronics Convention*. IEEE, 1102–1106.
[71] Jing Yu Koh, Daniel Fried, and Russ R. Salakhutdinov. 2024. Generating images with multimodal language models. *Advances in Neural Information Processing Systems* 36 (2024).
[72] Paweł Korus and Nasir Memon. 2022. Computational sensor fingerprints. *IEEE Transactions on Information Forensics and Security* 17 (2022), 2508–2523.
[73] Kalpesh Krishna, Yixiao Song, Marzena Karpinska, et al. 2023. Paraphrasing evades detectors of ai-generated text, but retrieval is an effective defense. *Advances in Neural Information Processing Systems* 36 (2023), 27469–27500.
[74] Weirui Kuang, Bingchen Qian, Zitao Li, et al. 2024. Federatedscope-llm: A comprehensive package for fine-tuning large language models in federated learning. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 5260–5271.
[75] Weirui Kuang, Bingchen Qian, Zitao Li, et al. 2024. Federatedscope-llm: A comprehensive package for fine-tuning large language models in federated learning. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 5260–5271.
[76] Rohith Kuditipudi, John Thickstun, Tatsunori Hashimoto, et al. 2023. Robust distortion-free watermarks for language models. arXiv:2307.15593. Retrieved from https://arxiv.org/abs/2307.15593
[77] Ashutosh Kumar, Sagarika Singh, Shiv Vignesh Murty, and Swathy Ragupathy. 2024. The Ethics of Interaction: Mitigating Security Threats in LLMs. arXiv:2401.12273. Retrieved from https://arxiv.org/abs/2401.12273
[78] Zhenzhong Lan, Mingda Chen, Sebastian Goodman, et al. 2019. Albert: A lite bert for self-supervised learning of language representations. arXiv:1909.11942. Retrieved from https://arxiv.org/abs/1909.11942
[79] Taesung Lee, Benjamin Edwards, Ian Molloy, et al. 2019. Defending against neural network model stealing attacks using deceptive perturbations. In *Proceedings of the 2019 IEEE Security and Privacy Workshops*. IEEE, 43–49.
[80] Taehyun Lee, Seokhee Hong, Jaewoo Ahn, et al. 2024. Who wrote this code? watermarking for code generation. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 4890–4911.
[81] Boquan Li, Mengdi Zhang, Peixin Zhang, et al. 2024. Resilient watermarking for llm-generated codes. arXiv:2402.07518. Retrieved from https://arxiv.org/abs/2402.07518
[82] Jiyao Li and Wei Liu. 2023. Summarization attack via paraphrasing (student abstract). In *Proceedings of the AAAI Conference on Artificial Intelligence*. 16250–16251.
[83] Linyang Li, Botian Jiang, Pengyu Wang, et al. 2023. Watermarking LLMs with weight quantization. In *Findings of the Association for Computational Linguistics*. 3368–3378.
[84] Shuai Li, Kejiang Chen, Kunsheng Tang, et al. 2023. FunctionMarker: Watermarking language datasets via knowledge injection. arXiv:2311.09535. Retrieved from https://arxiv.org/abs/2311.09535
[85] Sen Li, Ruochen Wang, Cho-Jui Hsieh, et al. 2024. Mulan: Multimodal-llm agent for progressive multi-object diffusion. arXiv:2402.12741. Retrieved from https://arxiv.org/abs/2402.12741
[86] Shen Li, Liuyi Yao, Jinyang Gao, et al. 2024. Double-I watermark: Protecting model copyright for LLM fine-tuning. arXiv:2402.14883. Retrieved from https://arxiv.org/abs/2402.14883
[87] Yinheng Li, Shaofei Wang, Han Ding, et al. 2023. Large language models in finance: A survey. In *Proceedings of the Fourth ACM International Conference on AI in Finance*. 374–382.
[88] Yuhang Li, Yihan Wang, Zhouxing Shi, et al. 2023. Improving the generation quality of watermarked large language models via word importance scoring. arXiv:2311.09668. Retrieved from https://arxiv.org/abs/2311.09668
[89] Yuanchun Li, Ziqi Zhang, Bingyan Liu, et al. 2021. ModelDiff: Testing-based DNN similarity comparison for model reuse detection. In *Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis*. 139–151.
[90] Zongjie Li, Chaozheng Wang, Pingchuan Ma, et al. 2023. On extracting specialized code abilities from large language models: A feasibility study. arXiv:2303.03012. Retrieved from https://arxiv.org/abs/2303.03012
[91] Zongjie Li, Chaozheng Wang, Shuai Wang, et al. 2023. Protecting intellectual property of large language model-based code generation apis via watermarks. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*. 2336–2350.
[92] Zhang Li, Biao Yang, Qiang Liu, et al. 2024. Monkey: Image resolution and text label are important things for large multi-modal models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 26763–26773.
[93] Jing Liang, Li Niu, Fengjun Guo, et al. 2021. Visible watermark removal via self-calibrated localization and background refinement. In *Proceedings of the 29th ACM International Conference on Multimedia*. Association for Computing Machinery, 4426–4434.

[94] Li Lin, Neeraj Gupta, Yue Zhang, et al. 2024. Detecting multimedia generated by large ai models: A survey. arXiv:2402.00045. Retrieved from https://arxiv.org/abs/2402.00045

[95] Xinyu Lin, Wenjie Wang, Yongqi Li, et al. 2024. Data-efficient fine-tuning for LLM-based recommendation. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 365–374.

[96] Aiwei Liu, Leyi Pan, Xuming Hu, et al. 2023. A semantic invariant robust watermark for large language models. In *Proceedings of the 12th International Conference on Learning Representations*.

[97] Aiwei Liu, Leyi Pan, Xuming Hu, et al. 2024. An unforgeable publicly verifiable watermark for large language models. In *Proceedings of the ICLR*.

[98] Aiwei Liu, Leyi Pan, Yijian Lu, et al. 2024. A survey of text watermarking in the era of large language models. *Comput. Surveys* 57, 2 (2024), 1–36.

[99] Yepeng Liu and Yuheng Bu. 2024. Adaptive text watermark for large language models. In *Proceedings of the 41st International Conference on Machine Learning*. 30718–30737.

[100] Yixin Liu, Hongsheng Hu, Xun Chen, et al. 2023. Watermarking Classification Dataset for Copyright Protection. arXiv:2305.13257. Retrieved from https://arxiv.org/abs/2305.13257

[101] Ning Lu, Shengcai Liu, Rui He, et al. 2023. Large language models can be guided to evade AI-generated text detection. arXiv:2305.10847. Retrieved from https://arxiv.org/abs/2305.10847

[102] Nils Lukas, Yuxuan Zhang, and Florian Kerschbaum. 2019. Deep neural network fingerprinting by conferrable adversarial examples. arXiv:1912.00888. Retrieved from https://arxiv.org/abs/1912.00888

[103] Aleksander Madry, Aleksandar Makelov, Ludwig Schmidt, et al. 2017. Towards deep learning models resistant to adversarial attacks. arXiv:1706.06083. Retrieved from https://arxiv.org/abs/1706.06083

[104] Elizabeth M. Merkhofer, John Henderson, Abigail S. Gertner, et al. 2022. Practical attacks on machine translation using paraphrase. In *Proceedings of the 15th Biennial Conference of the Association for Machine Translation in the Americas (Volume 1: Research Track)*. 227–239.

[105] Carmelo Militello, Leonardo Rundo, Salvatore Vitabile, et al. 2021. Fingerprint classification based on deep learning approaches: experimental findings and comparisons. *Symmetry* 13, 5 (2021), 750.

[106] Seyed-Mohsen Moosavi-Dezfooli, Alhussein Fawzi, and Pascal Frossard. 2016. Deepfool: A simple and accurate method to fool deep neural networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 2574–2582.

[107] Stephen Moskal, Sam Laney, Erik Hemberg, et al. 2023. LLMs Killed the Script Kiddie: How Agents Supported by Large Language Models Change the Landscape of Network Threat Testing. arXiv:2310.06936. Retrieved from https://arxiv.org/abs/2310.06936

[108] Sumeet Ramesh Motwani, Mikhail Baranchuk, Lewis Hammond, et al. 2023. A perfect collusion benchmark: How can AI agents be prevented from colluding with information-theoretic undetectability?. In *Proceedings of the Multi-Agent Security Workshop@ NeurIPS'23*.

[109] Travis Munyer and Xin Zhong. 2023. Deeptextmark: Deep learning based text watermarking for detection of large language model generated text. arXiv:2305.05773. Retrieved from https://arxiv.org/abs/2305.05773

[110] Pranav Narayanan Venkit. 2023. Towards a holistic approach: Understanding sociodemographic biases in NLP models using an interdisciplinary lens. In *Proceedings of the 2023 AAAI/ACM Conference on AI, Ethics, and Society*. 1004–1005.

[111] Leyi Pan, Aiwei Liu, Shiyu Huang, et al. 2025. Can LLM watermarks robustly prevent unauthorized knowledge distillation? arXiv:2502.11598. Retrieved from https://arxiv.org/abs/2502.11598

[112] Xudong Pan, Yifan Yan, Mi Zhang, et al. 2022. Metav: A meta-verifier approach to task-agnostic model fingerprinting. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 1327–1336.

[113] Qi Pang, Shengyuan Hu, Wenting Zheng, et al. 2024. Attacking LLM Watermarks by Exploiting Their Strengths. arXiv:2402.16187. Retrieved from https://arxiv.org/abs/2402.16187

[114] Guilherme Penedo, Quentin Malartic, Daniel Hesslow, Ruxandra Cojocaru, Hamza Alobeidli, Alessandro Cappelli, Baptiste Pannier, Ebtesam Almazrouei, and Julien Launay. 2023. The refinedweb dataset for falcon llm: Outperforming curated corpora with web data only. *Advances in Neural Information Processing Systems* 36 (2023), 79155–79172.

[115] Wenjun Peng, Jingwei Yi, Fangzhao Wu, et al. 2023. Are you copying my model? protecting the copyright of large language models for EaaS via backdoor watermark. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 7653–7668.

[116] Zirui Peng, Shaofeng Li, Guoxing Chen, et al. 2022. Fingerprinting deep neural networks globally via universal adversarial perturbations. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 13430–13439.

[117] Zhencan Peng, Zhizhi Wang, and Dong Deng. 2023. Near-duplicate sequence search at scale for large language model memorization evaluation. *Proc. ACM Manag. Data* 1, 2 (2023), 18 pages.

[118] Jipeng Qiang, Shiyu Zhu, Yun Li, et al. 2023. Natural language watermarking via paraphraser-based lexical substitution. *Artificial Intelligence* 317 (2023), 103859.
[119] Wenjie Qu, Dong Yin, Zixin He, et al. 2024. Provably robust multi-bit watermarking for AI-generated text via error correction code. arXiv:2401.16820. Retrieved from https://arxiv.org/abs/2401.16820
[120] Vyas Raina, Adian Liusie, and Mark Gales. 2024. Is LLM-as-a-judge robust? investigating universal adversarial attacks on zero-shot LLM assessment. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. 7499–7517.
[121] Ladislav Rampášek, Michael Galkin, Vijay Prakash Dwivedi, et al. 2022. Recipe for a general, powerful, scalable graph transformer. In *Proceedings of the Advances in Neural Information Processing Systems*. S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh (Eds.), Vol. 35, Curran Associates, Inc., 14501–14515.
[122] Jay Ratican and James Hutson. 2023. The six emotional dimension (6DE) model: A multidimensional approach to analyzing human emotions and unlocking the potential of emotionally intelligent artificial intelligence (AI) via large language models (LLM). *Journal of Artificial Intelligence and Robotics* 1, 1 (2023).
[123] Benjamin Reichman and Larry Heck. 2023. Cross-modal dense passage retrieval for outside knowledge visual question answering. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 2837–2842.
[124] Huali Ren, Anli Yan, Xiaojun Ren, et al. 2023. Ganfinger: Gan-based fingerprint generation for deep neural network ownership verification. arXiv:2312.15617. Retrieved from https://arxiv.org/abs/2312.15617
[125] Jie Ren, Han Xu, Pengfei He, et al. 2024. Copyright protection in generative AI: A technical perspective. arXiv:2402.02333. Retrieved from https://arxiv.org/abs/2402.02333
[126] Jie Ren, Han Xu, Yiding Liu, et al. 2024. A robust semantics-based watermark for large language model against paraphrasing. In *Findings of the Association for Computational Linguistics: NAACL 2024*. 613–625.
[127] Jonathan Roberts, Timo Lüddecke, Rehan Sheikh, et al. 2024. Charting new territories: Exploring the geographic and geospatial capabilities of multimodal llms. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 554–563.
[128] Vinu Sankar Sadasivan, Aounon Kumar, Sriram Balasubramanian, et al. 2023. Can ai-generated text be reliably detected? arXiv:2303.11156. Retrieved from https://arxiv.org/abs/2303.11156
[129] Ugur Sahin, Hang Li, Qadeer Khan, et al. 2024. Enhancing multimodal compositional reasoning of visual language models with generative negative mining. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*. 5563–5573.
[130] Gerard Salton and Chris Buckley. 1988. Term-weighting approaches in automatic text retrieval. *Inf. Process. Manag.* 24, 5 (1988), 513–523.
[131] Pedro Miguel Sánchez Sánchez, Jose Maria Jorquera Valero, Alberto Huertas Celdrán, et al. 2021. A survey on device behavior fingerprinting: Data sources, techniques, application scenarios, and datasets. *IEEE Communications Surveys and Tutorials* 23, 2 (2021), 1048–1077.
[132] Victor Sanh, Lysandre Debut, Julien Chaumond, et al. 2019. DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter. arXiv:1910.01108. Retrieved from https://arxiv.org/abs/1910.01108
[133] Sunandini Sanyal, Sravanti Addepalli, and R. Venkatesh Babu. 2022. Towards data-free model stealing in a hard label setting. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. IEEE, 15263–15272.
[134] Armin Sarabi, Tongxin Yin, and Mingyan Liu. 2023. An llm-based framework for fingerprinting internet-connected devices. In *Proceedings of the 2023 ACM on Internet Measurement Conference*. 478–484.
[135] Ryoma Sato, Yuki Takezawa, Han Bao, et al. 2023. Embarrassingly simple text watermarks. arXiv:2310.08920. Retrieved from https://arxiv.org/abs/2310.08920
[136] Ryoma Sato, Yuki Takezawa, Han Bao, Kenta Niwa, and Makoto Yamada. 2023. Embarrassingly simple text watermarks. arXiv:2310.08920. Retrieved from https://arxiv.org/abs/2310.08920
[137] Masoumeh Shafieinejad, Nils Lukas, Jiaqi Wang, Xinda Li, and Florian Kerschbaum. 2021. On the robustness of backdoor-based watermarking in deep neural networks. In *Proceedings of the 2021 ACM Workshop on Information Hiding and Multimedia Security*. 177–188.
[138] Peter Shaw, Jakob Uszkoreit, and Ashish Vaswani. 2018. Self-attention with relative position representations. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 2 (Short Papers)*. Association for Computational Linguistics, 464–468.
[139] Xicong Shen, Yang Liu, Huiqi Liu, et al. 2023. A split-and-privatize framework for large language model fine-tuning. arXiv:2312.15603. Retrieved from https://arxiv.org/abs/2312.15603
[140] Han Shi, Jiahui Gao, Xiaozhe Ren, et al. 2021. SparseBERT: Rethinking the importance analysis in self-attention. In *Proceedings of the 38th International Conference on Machine Learning (Proceedings of Machine Learning Research)*. Marina Meila and Tong Zhang (Eds.), PMLR, 9547–9557.
[141] Zhouxing Shi, Yihan Wang, Fan Yin, et al. 2024. Red teaming language model detectors with language models. *Transactions of the Association for Computational Linguistics* 12 (2024), 174–189.

[142] Karanpartap Singh and James Zou. 2023. New evaluation metrics capture quality degradation due to LLM watermarking. arXiv:2312.02382. Retrieved from https://arxiv.org/abs/2312.02382

[143] Naman Deep Singh, Francesco Croce, and Matthias Hein. 2023. Revisiting adversarial training for imagenet: Architectures, training and generalization across threat models. *Advances in Neural Information Processing Systems* 36 (2023), 13931–13955.

[144] Jiao Sun, Xuezhe Ma, and Nanyun Peng. 2021. AESOP: Paraphrase generation with adaptive syntactic control. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*. 5176–5189.

[145] Ruizhou Sun, Yukun Su, and Qingyao Wu. 2023. Denet: Disentangled embedding network for visible watermark removal. In *Proceedings of the AAAI Conference on Artificial Intelligence*. 2411–2419.

[146] Xiaofei Sun, Xiaoya Li, Yuxian Meng, et al. 2023. Defending against backdoor attacks in natural language generation. In *Proceedings of the AAAI Conference on Artificial Intelligence*. 5257–5265.

[147] Zhensu Sun, Xiaoning Du, Fu Song, et al. 2022. Coprotector: Protect open-source code against unauthorized training usage with data poisoning. In *Proceedings of the ACM Web Conference 2022*. 652–660.

[148] Zhensu Sun, Xiaoning Du, Fu Song, et al. 2023. CodeMark: Imperceptible Watermarking for Code Datasets against Neural Code Completion Models. In *Association for Computing Machinery*. 1561–1572.

[149] Zhensu Sun, Xiaoning Du, Fu Song, and Li Li. 2023. Codemark: Imperceptible watermarking for code datasets against neural code completion models. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 1561–1572.

[150] Yuki Takezawa, Ryoma Sato, Han Bao, Kenta Niwa, and Makoto Yamada. 2023. Necessary and sufficient watermark for large language models. *Transactions on Machine Learning Research* (2023).

[151] Leonard Tang, Gavin Uberti, and Tom Shlomi. 2023. Baselines for identifying watermarked large language models. arXiv:2305.18456. Retrieved from https://arxiv.org/abs/2305.18456

[152] Ruixiang Tang, Qizhang Feng, Ninghao Liu, et al. 2023. Did you train on my dataset? towards public dataset protection with cleanlabel backdoor watermarking. *ACM SIGKDD Explorations Newsletter* 25, 1 (2023), 43–53.

[153] Raphael Tang, Crystina Zhang, Xueguang Ma, et al. 2024. Found in the middle: Permutation self-consistency improves listwise ranking in large language models. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. 2327–2340.

[154] Hugo Touvron, Louis Martin, Kevin Stone, et al. 2023. Llama 2: Open Foundation and Fine-Tuned Chat Models. arXiv:2307.09288. Retrieved from https://arxiv.org/abs/2307.09288

[155] Cheng-Hao Tu, Huei-Fang Yang, Shih-Min Yang, et al. 2021. SemanticHash: Hash coding via semantics-guided label prototype learning. *IEEE Transactions on Artificial Intelligence* 2, 1 (2021), 42–57.

[156] Marco Uras, Enrico Ferrara, Raimondo Cossu, et al. 2022. MAC address de-randomization for WiFi device counting: Combining temporal-and content-based fingerprints. *Computer Networks* 218 (2022), 109393.

[157] Ashish Vaswani, Prajit Ramachandran, Aravind Srinivas, et al. 2021. Scaling local self-attention for parameter efficient visual backbones. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 12894–12904.

[158] Ashish Vaswani, Noam Shazeer, Niki Parmar, et al. 2017. Attention is all you need. In *Proceedings of the Advances in Neural Information Processing Systems*. Curran Associates, Inc.

[159] Gaurav Verma, Minje Choi, Kartik Sharma, et al. 2024. Mysterious projections: Multimodal LLMs gain domain-specific visual capabilities without richer cross-modal projections. arXiv:2402.16832. Retrieved from https://arxiv.org/abs/2402.16832

[160] Nalin Wadhwa, Jui Pradhan, Atharv Sonwane, et al. 2024. Core: Resolving code quality issues using llms. *Proceedings of the ACM on Software Engineering* 1, FSE (2024), 789–811.

[161] Jingtan Wang, Xinyang Lu, Zitong Zhao, et al. 2023. WASA: Watermark-based source attribution for large language model-generated data. arXiv:2310.00646. Retrieved from https://arxiv.org/abs/2310.00646

[162] Lean Wang, Wenkai Yang, Deli Chen, et al. 2023. Towards codable text watermarking for large language models. arXiv:2307.15992. Retrieved from https://arxiv.org/abs/2307.15992

[163] Run Wang, Haoxuan Li, Lingzhou Mu, et al. 2022. Rethinking the vulnerability of dnn watermarking: Are watermarks robust against naturalness-aware perturbations?. In *Proceedings of the 30th ACM International Conference on Multimedia*. 1808–1818.

[164] Si Wang and Chip-Hong Chang. 2021. Fingerprinting deep neural networks-a deepfool approach. In *Proceedings of the 2021 IEEE International Symposium on Circuits and Systems*. IEEE, 1–5.

[165] Tan Wang, Kevin Lin, Linjie Li, et al. 2023. Equivariant similarity for vision-language foundation models. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 11998–12008.

[166] Yi Wang, Yinan He, Yizhuo Li, et al. 2023. Internvid: A large-scale video-text dataset for multimodal understanding and generation. arXiv:2307.06942. Retrieved from https://arxiv.org/abs/2307.06942

[167] Yulong Wang, Xiaolu Zhang, Lingxi Xie, et al. 2020. Pruning from Scratch. In *Proceedings of the 34th AAAI Conference on Artificial Intelligence*. AAAI Press, 12273–12280.

[168] Zhou Wang, Alan C. Bovik, Hamid R. Sheikh, et al. 2004. Image quality assessment: from error visibility to structural similarity. *IEEE Transactions on Image Processing* 13, 4 (2004), 600–612.
[169] Zezhong Wang, Fangkai Yang, Lu Wang, et al. 2024. SELF-GUARD: Empower the LLM to safeguard itself. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. 1648–1668.
[170] Johnny Tian-Zheng Wei, Ryan Yixiang Wang, and Robin Jia. 2024. Proving membership in LLM pretraining data via data watermarks. arXiv:2402.10892. Retrieved from https://arxiv.org/abs/2402.10892
[171] Thomas Wolf, Lysandre Debut, Victor Sanh, et al. 2020. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. Qun Liu and David Schlangen (Eds.), Association for Computational Linguistics, Online, 38–45.
[172] Bram Wouters. 2024. Optimizing watermarks for large language models. In *Proceedings of the 41st International Conference on Machine Learning*. 53251–53269.
[173] Bingyang Wu, Yinmin Zhong, Zili Zhang, et al. 2023. Fast Distributed Inference Serving for Large Language Models. arXiv:2305.05920. Retrieved from https://arxiv.org/abs/2305.05920
[174] Shengqiong Wu, Hao Fei, Leigang Qu, et al. 2024. Next-gpt: Any-to-any multimodal llm. In *Proceedings of the 41st International Conference on Machine Learning*.
[175] Yihan Wu, Zhengmian Hu, Hongyang Zhang, et al. 2023. Dipmark: A stealthy, efficient and resilient watermark for large language models. arXiv:2310.07710. Retrieved from https://arxiv.org/abs/2310.07710
[176] Bin Xia, Shiyin Wang, Yingfan Tao, et al. 2024. Llmga: Multimodal large language model based generation assistant. In *Proceedings of the European Conference on Computer Vision*. Springer, 389–406.
[177] Jiashu Xu, Fei Wang, Mingyu Ma, et al. 2024. Instructional fingerprinting of large language models. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. 3277–3306.
[178] Mengwei Xu, Wangsong Yin, Dongqi Cai, et al. 2024. A Survey of Resource-efficient LLM and Multimodal Foundation Models. arXiv:2401.08092. Retrieved from https://arxiv.org/abs/2401.08092
[179] Weilin Xu, David Evans, and Yanjun Qi. 2017. Feature squeezing: Detecting adversarial examples in deep neural networks. arXiv:1704.01155. Retrieved from https://arxiv.org/abs/1704.01155
[180] Fuzhao Xue, Yao Fu, Wangchunshu Zhou, et al. 2023. To repeat or not to repeat: Insights from scaling llm under token-crisis. *Advances in Neural Information Processing Systems* 36 (2023), 59304–59322.
[181] Mingfu Xue, Xin Wang, Yinghao Wu, et al. 2022. InFIP: An explainable DNN intellectual property protection method based on intrinsic features. arXiv:2210.07481. Retrieved from https://arxiv.org/abs/2210.07481
[182] Vikas Yadav, Zheng Tang, and Vijay Srinivasan. 2024. Pag-llm: Paraphrase and aggregate with large language models for minimizing intent classification errors. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 2569–2573.
[183] Borui Yang, Wei Li, Liyao Xiang, et al. 2023. Towards code watermarking with dual-channel transformations. arXiv:2309.00860. Retrieved from https://arxiv.org/abs/2309.00860
[184] Kang Yang, Run Wang, and Lina Wang. 2022. Metafinger: Fingerprinting the deep neural networks with meta-training. In *Proceedings of the 31st International Joint Conference on Artificial Intelligence*.
[185] Xianjun Yang, Wei Cheng, Yue Wu, et al. 2024. DNA-GPT: Divergent N-Gram analysis for training-free detection of GPT-generated text. In *Proceedings of the ICLR*.
[186] Xinyu Yang and Jizhe Zhou. 2024. Research about the Ability of LLM in the Tamper-Detection Area. arXiv:2401.13504. Retrieved from https://arxiv.org/abs/2401.13504
[187] Xiaoyu Ye, Hao Huang, Jiaqi An, et al. [n. d.]. DUAW: Data-free universal adversarial watermark against stable diffusion customization. In *Proceedings of the ICLR 2024 Workshop on Secure and Trustworthy Large Language Models*.
[188] Fangcong Yin, Xi Ye, and Greg Durrett. 2024. Lofit: Localized fine-tuning on llm representations. *Advances in Neural Information Processing Systems* 37 (2024), 9474–9506.
[189] Zhenfei Yin, Jiong Wang, Jianjian Cao, Zhelun Shi, Dingning Liu, Mukai Li, Xiaoshui Huang, Zhiyong Wang, Lu Sheng, Lei Bai, et al. 2023. LAMM: Language-assisted multi-modal instruction-tuning dataset, framework, and benchmark. In *Proceedings of the 37th International Conference on Neural Information Processing Systems*. 26650–26685.
[190] KiYoon Yoo, Wonhyuk Ahn, Jiho Jang, et al. 2023. Robust multi-bit natural language watermarking through invariant features. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, Toronto, Canada, 2092–2115.
[191] Ning Yu, Vladislav Skripniuk, Sahar Abdelnabi, et al. 2021. Artificial fingerprinting for generative models: Rooting deepfake attribution in training data. In *Proceedings of the IEEE/CVF International conference on computer vision*. 14448–14457.
[192] Tongxin Yuan, Zhiwei He, Lingzhong Dong, et al. 2024. R-Judge: Benchmarking safety risk awareness for LLM agents. In *Findings of the Association for Computational Linguistics*. 1467–1490.

[193] Zhenrui Yue, Zhankui He, Huimin Zeng, et al. 2021. Black-box attacks on sequential recommenders via data-free model extraction. In *Proceedings of the 15th ACM Conference on Recommender Systems*. 44–54.
[194] Or Zamir. 2024. Excuse me, sir? Your language model is leaking (information). arXiv:2401.10360. Retrieved from https://arxiv.org/abs/2401.10360
[195] Boyi Zeng, Lizheng Wang, Yuncong Hu, et al. 2024. Huref: Human-readable fingerprint for large language models. *Advances in Neural Information Processing Systems* 37 (2024), 126332–126362.
[196] Jun Zhan, Junqi Dai, Jiasheng Ye, et al. 2024. AnyGPT: Unified multimodal LLM with discrete sequence modeling. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 9637–9662.
[197] Hanlin Zhang, Benjamin L. Edelman, Danilo Francati, Daniele Venturi, Giuseppe Ateniese, and Boaz Barak. 2024. Watermarks in the sand: Impossibility of strong watermarking for generative models. In *Proceedings of the ICLR 2024 Workshop on Secure and Trustworthy Large Language Models*.
[198] Jie Zhang, Dongdong Chen, Jing Liao, et al. 2021. Deep model intellectual property protection via deep watermarking. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 44, 8 (2021), 4005–4020.
[199] Ruisi Zhang, Shehzeen Samarah Hussain, Paarth Neekhara, et al. 2024. {REMARK-LLM}: A robust and efficient watermarking framework for generative large language models. In *Proceedings of the 33rd USENIX Security Symposium*. 1813–1830.
[200] Xinghua Zhang, Bowen Yu, Haiyang Yu, et al. 2023. Wider and deeper llm networks are fairer llm evaluators. arXiv:2308.01862. Retrieved from https://arxiv.org/abs/2308.01862
[201] Jiawei Zhao, Kejiang Chen, Xiaojian Yuan, et al. 2024. Silent guardian: Protecting text from malicious exploitation by large language models. *IEEE Transactions on Information Forensics and Security* (2024).
[202] Jingjing Zhao, Qingyue Hu, Gaoyang Liu, et al. 2020. AFA: Adversarial fingerprinting authentication for deep neural networks. *Computer Communications* 150 (2020), 488–497.
[203] Lei Zhao, Yunfeng Zou, Chao Xu, Yulong Ma, et al. 2024. Robust soliton distribution-based zero-watermarking for semi-structured power data. *Electronics* 13, 3 (2024), 655.
[204] Xuandong Zhao, Prabhanjan Ananth, Lei Li, et al. 2023. Provable robust watermarking for ai-generated text. arXiv:2306.17439. Retrieved from https://arxiv.org/abs/2306.17439
[205] Xuandong Zhao, Lei Li, and Yu-Xiang Wang. 2024. Permute-and-flip: An optimally stable and watermarkable decoder for LLMs. In *Proceedings of the Neurips Safe Generative AI Workshop*.
[206] Xuandong Zhao, Yu-Xiang Wang, and Lei Li. 2023. Protecting language generation models via invisible watermarking. In *Proceedings of the International Conference on Machine Learning*. PMLR, 42187–42199.
[207] Huadi Zheng, Qingqing Ye, Haibo Hu, et al. 2022. Protecting decision boundary of machine learning model with differentially private perturbation. *IEEE Trans. Dependable Secur. Comput.* 19, 3 (2022), 2007–2022.
[208] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, et al. [n. d.]. LMSYS-chat-1M: A large-scale real-world LLM conversation dataset. In *Proceedings of the 12th International Conference on Learning Representations*.
[209] Yue Zheng, Si Wang, and Chip-Hong Chang. 2022. A dnn fingerprint for non-repudiable model ownership identification and piracy detection. *IEEE Transactions on Information Forensics and Security* 17 (2022), 2977–2989.
[210] Kun Zhou, Yutao Zhu, Zhipeng Chen, et al. 2023. Don't Make Your LLM an Evaluation Benchmark Cheater. arXiv:2311.01964. Retrieved from https://arxiv.org/abs/2311.01964
[211] Xiaoyi Zhou, Yue Ma, Qingquan Zhang, et al. 2021. A reversible watermarking system for medical color images: Balancing capacity, imperceptibility, and robustness. *Electronics* 10, 9 (2021).
[212] Han Zhu, Mingsheng Long, Jianmin Wang, et al. 2016. Deep hashing network for efficient similarity retrieval. In *Proceedings of the AAAI Conference on Artificial Intelligence*.
[213] Terry Yue Zhuo, Yujin Huang, Chunyang Chen, et al. 2023. Red Teaming ChatGPT via Jailbreaking: Bias, Robustness, Reliability and Toxicity. arXiv:2301.12867. Retrieved from https://arxiv.org/abs/2301.12867
[214] Andy Zou, Zifan Wang, Nicholas Carlini, et al. 2023. Universal and Transferable Adversarial Attacks on Aligned Language Models. arXiv:2307.15043. Retrieved from https://arxiv.org/abs/2307.15043

Received 14 February 2025; revised 16 July 2025; accepted 13 October 2025